Appendices

The appendices provide supplementary resources to help readers reinforce their understanding of SysML v2 and apply the concepts covered throughout the book. They include exercises for hands-on practice, a quick reference to SysML v2 textual notation, and a comparison of SysML v1 and SysML v2 to help readers understand the differences between the two versions.

The appendices include the following:

- *Appendix A, SysML v2 Text Notation QuickGuide*
- *Appendix B, SysML v1 vs. SysML v2*
- *Appendix C, Exercises*

A

SysML v2 Text Notation Quick Guide

This appendix is meant to provide a quick guide to the syntax of commonly used SysML v2 textual statements.

Constructs and expressions

Construct	Expression	Example
Line Comment	// <text>	// this is a comment
Block Comment	/* ... */	/* multi-line note */
Documentation	doc /* ... */	doc /* engine description */
Definition	<kind> def <name>	part def Rover
Usage	<name> : <type>	part wheel: Wheel
Subsetting	<name> : <type> subsets <other>	connection link: Link subsets allLinks
Redefinition	<name> : <type> redefines <other>	attribute mass redefines baseMass
Import	<visibility> import <package>	public import SI::*
Multiplicity	<element>[<range>]	part sensor[1..4]: Sensor
Assignment	assign <name> := <expr>	assign weight := 35[kg]
Default assignment	<name>: <type> default := <value>	attribute torque: Real default := 0
Binding	<name> = <expr>	attribute id = 42

Construct	Expression	Example
Default binding	<name>: <type> = <value>	attribute rpm : Real default = 40;
Feature reference	<element>.<feature>	Car.milesTravelled
Qualified Reference	<namespace>::<member>	ScalarValues::Real; ISQ::LengthValue.
Comparison operators	<, <=, ==, !=, >, >=	measuredTorque <= maxTorque;
Metadata Definition	metadata def <element>	metadata def QuantifiedValue { digitsOfPrecision: Integer; }
Metadata Usage	@<metadata>	@QuantifiedValue { digitsOfPrecision = 6; }

Table A.1 – Constructs and expressions

Relations table

Keyword	Short Form
defined by	:
specializes	:>
subsets	:>
references	::>
crosses	=>
redefines	:>>
depends	dependency from a to b;

Table A.2 – Relations

Operators

Relational operators

Relational operators compare two values and produce a Boolean result.

Operator	Meaning
==	equal to
!=	not equal
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal

Table A.3 – Relational operators

Arithmetic operators

Operator	Meaning
+	addition
-	subtraction
*	multiplication
/	division
%	modulo (remainder)
^	exponentiation

Table A.4 – Arithmetic operators

Logical operators

Operator	Meaning
and	logical and (short-circuit)
or	logical or
not	logical negation
xor	exclusive or

Operator	Meaning
&	inclusive and (non-short-circuit)

Table A.5 – Logical operators

Occurrences

Category	Example
Occurrence Definition	occurrence def Mission
Specialization	occurrence def SecretMission specializes Mission;
Occurrence Usage	occurrence missionX: Mission;

Table A.6 – Occurrences

Items

Category	Example
Item Definition	item def Telemetry { attribute value: Real; attribute unit: String; }
Specialization	item def satelliteTelemetry specializes Telemetry;
Item Usage	item satData: satelliteTelemetry

Table A.7 – Items

Parts

Category	Example
Part Definition	part def Engine { attribute rpm: Integer; }
Specialization	part def SportEngine specializes Engine { attribute turbo: Boolean; }
Part Usage	part vehicle { part eng: Engine; }

Category	Example
Requirement Satisfaction	satisfy requirement75;

Table A.8 – Parts

Connections

Category	Example
Connection Definition	connection def DataLink { end source; end target; attribute bw : Real; }
Connection Usage	connection uplink: DataLink { end ::> cpu; end ::> radio; }
Interface Definition	interface def CommInterface { end p1: P1; end p2:P2; }
Interface Usage	interface ci: CommInterface connect part1.mp1 to part2.p2
Allocation Definition	allocation def L2P { end LSide; end PSide; }
Allocation Usage	allocation l2p: L2P allocate a to s;
Succession Usage	succession first actionA then actionB;
Succession Flow Usage	succession flow src.output to tgt.input;
Flow Definition	flow def PayloadTransfer { ... };
Flow Usage	flow src.output to tgt.input;

Table A.9 – Connections

Ports

Category	Example
Port Definition	port def TelemetryPort { in item tIn : Telemetry; out item tOut : Telemetry; }
Port Usage	port p: TelemetryPort

Table A.10 – Ports

Requirements

Category	Example
Requirement Definition	requirement def MaxSpeed { attribute limit: Real; }
Requirement Usage	requirement enforceSpeed : MaxSpeed { :>> limit = 120.0; }
Requirement Property — subject	subject target: Aircraft;
Requirement Property — actor	actor pilot: Person;
Requirement Property – stakeholder	stakeholder stakeholderPerson : Person;
Requirement Property — require constraint	require constraint { speed <= limit };

Table A.11 – Requirements

Constraints

Category	Example
Constraint Definition	constraint def SpeedLimit { in speed : Real; in max : Real; speed <= max }
Constraint Usage	constraint { battery.level > 20 }

Table A.12 – Constraints

Calculations

Category	Example
Calculation Definition	calc def PowerCalc { in v: Real; in i: Real; return p : Real; v * i }
Calculation Usage	calc computePower: PowerCalc{ in v = voltage; in i: current; } attribute powerOut : Real = PowerCalc(voltage, current);

Table A.13 – Calculations

Actions

Category	Example
Action Definition	action def StartUp { in enable: Boolean; out status: Integer; }
Perform Action	perform action boot: StartUp;

Composite Action	action def Drive {
	in throttle : Real;
	action accelerate;
	then action cruise;
	}

Table A.14 – Actions

States

Category	Example
State Definition	state def Mode { state Idle; state Active; }
State Usage	state motorMode : Mode;

Table A.15 – States

Cases

Category	Example
Case Definition	case def StartupCase { tduration: TimeValue; }
Case Usage	case run: StartupCase { tduration = 5[s]; }

Category	Example

Use Case Definition	use case def AuthenticateUser {
	in username: String;
	in password: String;
	}

Use Case Usage	use case login: AuthenticateUser {
	username = "guest";
	}

Analysis Case Definition	analysis def ThermalAnalysis {
	in ambient: TemperatureValue;
	}

Analysis Case Usage	analysis tAnalysis : ThermalAnalysis {
	ambient = 25['°C'];
	}

Verification Case Definition	verification def LoadTest {
	in maxLoad : ForceValue;
	}

Verification Case Usage	verification loadCheck : LoadTest {
	maxLoad = 500[N];
	}

Case Property: subject	subject drone: DeliveryDrone
Case Property: actor	actor operator: Person

Case Property: objective	objective {
	doc /* achieve stability during launch */
	}

Table A.16 – Cases

B

SysML v1 vs. SysML v2

SysML v2 is not "SysML v1 with a few additions." It is a new language architecture with a different kernel, a different semantic strategy, and a different relationship between models, notations, and tools. That difference matters most for people who already *know* SysML v1: it is easy to assume familiar constructs exist somewhere in SysML v2, only renamed. Sometimes that is true. Often it is not.

SysML v1 mixes *declarative* and *imperative* modeling styles, largely because it inherits both structural, constraint-oriented concepts and executable behavioral notations from UML. A *declarative* statement specifies what must be true (properties, relationships, invariants, or constraints) without prescribing the steps used to achieve them. An *imperative* statement specifies what is done and in what order: actions, control flow, and operational sequencing. In SysML v1, both styles appear side by side: blocks and requirements often read declaratively, while activities, state machines, and interactions frequently read imperatively, with much of the *how* embedded in the behavioral notations.

SysML v2 adopts a more uniform foundation by making the language *entirely declarative*: models describe elements, relations, and constraints rather than embedding procedural semantics in a diagram grammar. This makes it easier to define rigorous, formal semantics and to support consistent tooling and automation, but it also raises a practical challenge: engineers still need to express behaviors that are naturally *imperative*. SysML v2 addresses that need by representing *imperative intent* declaratively, using explicit model elements and relations for sequencing and flow (for example, action usages linked by successions and declared value flows). This foundational shift (imperative meaning expressed through declarative structure) drives many of the specific differences between SysML v1 and SysML v2.

This appendix highlights the practical differences that affect day-to-day modeling: requirements and traceability, the elaboration of cases (use, analysis, verification), structural modeling, the textual notation as the primary representation and what that enables, and the shift from *diagram types* to definable views. The intent is not nostalgia or a checklist; it is to clarify how to think in SysML v2 when your instincts were shaped by SysML v1.

SysML v1 and SysML v2: Different foundations

SysML v1 is a UML profile. All of its semantics are inherited from UML, and all of its SysML concepts are introduced via UML extension mechanisms – profiles, stereotypes, tagged values, and model libraries. As a result, like UML, SysML v1 is a graphical language with no defined textual representation. The primary

interaction with the model is through diagram editors, and the model is frequently explained in terms of the canonical diagram set (bdd, ibd, act, sd, stm, req, par, uc, pkg, plus tool tables/matrices).

SysML v2 is an entirely new language stack built on KerML and a SysML layer on top of that kernel. Its design goal is increased rigor and regularity: fewer special cases, a more uniform set of rules, and first-class support for analysis and automation. In practice, the biggest cognitive shift is that SysML v2 is *text-first*: diagrams remain important, but they are *views* of the model, not the model itself. Everything in SysML can be represented in the textual notation, but not all of SysML v2 has a corresponding graphical visualization.

That foundational choice has ripple effects:

- You model with a uniform definition–usage pattern across most concepts.
- You can treat requirements, cases, actions, constraints, and even views as real model elements that can be authored, queried, versioned, and reused.
- Tool integration is not an afterthought; the language and its surrounding standard ecosystem assume models participate in automation and multi-tool workflows.

Requirements: From diagram-centric traceability to executable constraint-bearing elements

Requirements in SysML v2 are significantly different from requirements in SysML v1.

Requirements in SysML v1

In SysML v1, requirements are primarily captured as a specialized element (a stereotyped UML *Class*) whose core payload is a textual statement plus a few conventional properties such as id and text. The *Requirements Diagram* is specifically defined to capture requirement hierarchies and to relate requirements to other model elements via relationships such as *trace*, *deriveReqt*, *satisfy*, and *verify*.

This is effective for traceability. It is less effective for precision. A v1 requirement usually *describes* a constraint in text ("shall be between X and Y"), but it is not itself a constraint expression that a tool can evaluate without additional conventions or tooling rules. Parametric constraints can provide analyzable structure, but those constraints often live "next to" requirements rather than *inside* them.

Requirements in SysML v2

SysML v2 treats a requirement as a kind of constraint-bearing model element with explicit structure:

- A *subject* that identifies what the requirement applies to
- *Documentation* (normative prose) to communicate intent
- *Assumptions* that scope when the requirement is applicable
- *Required constraints* that can be expressed as evaluable expressions
- Optional metadata annotations (risk, status, authoring info, etc.)

The result is that a SysML v2 requirement can be both *communicative* (via doc) and *computable* (via explicit constraint expressions). This changes how requirements integrate with design models:

- Satisfaction can point to specific features that demonstrate compliance (the model can be structured so "what satisfies this" is explicit and reviewable).

- Requirements can be written to support automated checking without forcing you into a separate parametric "side model."
- Assumptions move from informal text into explicit applicability conditions, so the model can represent conditional requirements without burying the condition in prose.

Functional differences that matter

Here are the differences that most strongly affect modeling practice:

- **Applicability is modelable, not merely documented.**
- SysML v1 often encodes "if…" conditions in prose. SysML v2 allows explicit assumptions (for example, *only if gasType is Oxygen*), so the requirement is unambiguous about when it is intended to hold.
- **Requirements can directly host evaluable constraints.**
- Instead of a requirement saying "shall be ≤ 2000 W" and then hoping an engineer encodes that constraint somewhere else, SysML v2 can include an explicit constraint expression that can be evaluated.
- **Satisfaction and verification connect to richer semantic artifacts.**
- SysML v1's *satisfy* and *verify* are traceability relationships, typically used on requirement diagrams. In v2, they are statements within parts and verification cases, respectively.
- **Trace is not defined in SysML v2**
- The *trace* relation in SysML v1 is removed, so a general dependency is used instead.

SysML v2 retains the intent (linking requirements to design and evidence), but the surrounding language gives you more structured places to *put* the design rationale and evidence mechanisms, especially through cases (see the next section).

Cases: Use cases, analysis cases, and verification cases as first-class modeling constructs

SysML v1 has the notion of use cases. SysML v2 generalizes the concept to *cases* (special kinds of computations), and then specializes the concept into use cases, analysis cases, and verification cases.

SysML v1: Use cases and tests as separate worlds

SysML v1 inherits UML Use Cases and provides the requirement relationships needed for traceability. But *analysis* and *verification* are usually modeled indirectly:

- Analysis is often represented with parametric constraints, activities, external simulation models, or tool-specific integrations.
- Verification is frequently documented outside the SysML model, or captured using test-case stereotypes/tool profiles, or expressed in activity diagrams with conventions.

So, in v1, use cases are a canonical modeling element; analysis and verification are commonly modeled via patterns rather than standardized language constructs.

SysML v2: Cases unify intent, inputs, and outcomes

SysML v2 makes "case" a core concept: a case is defined relative to a *subject*, has an *objective*, and may have actor parameters. Specialized cases give the language a consistent way to represent:

- *Use cases*: Stakeholder-visible outcomes, often relating to requirements.
- *Analysis cases*: Evaluative computations/constraints (performance, safety, trade-offs).
- *Verification cases*: Explicit verification intent, behavior, and a verdict.

The practical effect is that requirements no longer need to carry all the burden of "what do we do with this requirement?" Cases give you explicit, standardized containers for:

- The operational context of a requirement (use case)
- The evaluation logic that demonstrates design adequacy (analysis case)
- The verification intent and evidence-producing behavior (verification case)

Why "cases" are distinct from requirements and parts

It helps to separate three roles that are often blurred in SysML v1 practice:

- A *part/structure* describes *what exists* and how it is composed.
- A *requirement* states *what must be true* about a subject (possibly under assumptions).
- A *case* defines *how you reason about* a subject to achieve an objective, whether by operational interaction (use), evaluation (analysis), or evidence generation (verification).

In other words: requirements express *necessary properties*; cases express *purposeful reasoning or demonstration* about those properties.

Structural modeling: From blocks and proxy ports to a regular definition—usage language

SysML v1 stereotypes the UML *Class* metaclass to create blocks as the key structural element. With SysML v2, the structural model is enhanced to include occurrences (things that have temporal and spatial extents), *items* (occurrences that represent things that are acted upon), and *parts* (things that can act).

SysML v1 structural practice

In SysML v1, the core structural element is the *Block*, with:

- Value properties typed by value types
- Part properties typed by blocks
- Ports (standard ports, flow ports in older versions, proxy ports), interface blocks, and item flows
- Connectors (with stereotypes such as binding connectors) and numerous patterns for interfaces and flows

This works well, but it is a patchwork of UML ancestry and SysML extensions. Many concepts exist because the language had to "fit" inside UML.

In SysML v1, an *interface* is fundamentally a UML concept: it is a *classifier* that specifies a contract, an agreed set of operations, receptions, and/or signals (and, by convention in SysML practice, sometimes data items) that a

block or component *provides* or *requires*. In other words, it expresses an agreement about *services and obligations* independent of the physical or structural details of how two parts are connected. When modelers use provided/required interfaces on ports in SysML v1, the interface is primarily describing *behavioral compatibility* ("what can be called or exchanged"), and connectors are then used to wire those ports together. With the introduction of v1 *proxy ports,* the *interface* concept was elaborated into *interface blocks*, which still had the same purpose: to specify a contract between blocks.

SysML v2 structural practice

SysML v2 aims for regularity. You see that immediately in the pervasive split between:

- *Definitions* (part def, attribute def, action def, connection def, …), and
- *Usages* (part, attribute, action, connection, …)

This regularity reduces "special case learning," but it also changes how you read models:

- When you see def, you are introducing a reusable element (a type-like construct).
- When you do not see def, you are using it in context (an instance role-like construct).

SysML v2 also provides clearer separation among:

- Structural topology (definitions/usages)
- Runtime state and execution (occurrences)
- And analysis snapshots/time slices (modeling constructs intended to support reasoning about time and execution state)

Ports and connections: Semantics vs. diagram conventions

SysML v2 makes ports and connections part of the same "definition/usage" discipline:

- Ports are owned connection points with typed, directed features.
- Compatibility includes *conjugation*: two ports can conform to the same port definition, but one is the conjugate of the other so directions align.

Connections are first-class as well:

- A *connection definition* captures the semantics of an interconnection (binding, flow, succession, interface semantics, and so on).
- A *connection usage* instantiates that definition between specific features (ports, parts, actions).
- Connections can carry attributes and constraints, rather than forcing you to wrap a connector in auxiliary structure just to annotate it.

In SysML v2, an *interface* is treated as part of the connection semantic space: it is a specialized form of interconnection specification, aligned with the *connection definition/usage* pattern, rather than a standalone classifier whose primary purpose is behavioral service contracting. Practically, the interface in SysML v2 is used to characterize the *semantics of the interconnection itself (what* can flow, bind, or interact across that link), and it is applied in the context of a connection (and often ports) that already makes the structural collaboration explicit. So, the emphasis shifts from "a type that lists services" (v1) to "a connection-level contract that constrains and explains interaction across the link" (v2).

This is a substantive departure from v1, where connectors are visually prominent but semantically often *thin* unless augmented by patterns.

Behavioral modeling in SysML v1 vs. SysML v2

The two versions of SysML differ in behavioral modeling in two primary ways. First, the declarative nature of v2 necessitates the explicit declaration of sequencing, which is not required in v1. Second, the expressiveness of state machines is reduced with the removal of all pseudostates.

Behavioral modeling in SysML v1

SysML v1 behavioral modeling is completely inherited from UML, so it is expressed through the UML families of *activities, interactions*, and *state machines*, with SysML usage patterns layered on top. In practice, that means behavior is often authored and reviewed through diagrams, and teams rely on diagram notation (and tool behavior) to convey ordering, branching, and concurrency.

- *Interactions* in SysML v1 are UML interactions (typically sequence diagrams): lifelines, messages, and combined fragments (alt/opt/loop, and so on) capture scenario order, conditionality, and repetition. This can be very expressive, but it is also diagram-grammar-heavy and can vary in consistency across tools and teams.
- *Actions* are usually modeled as UML activities: action nodes connected by control flows and object flows, mediated by pins and control nodes (fork/join/merge/decision). The notation supports complex flow logic, but it introduces many specialized constructs and can encourage *diagram-centric* semantics.
- *State machines* are UML state machines, including a large catalog of *pseudostates* (initial, choice, junction, history, entry/exit points, forks/joins, and others). Pseudostates allow compact expression of complex transition logic, but they significantly increase language surface area and raise interoperability risk because tools may implement edge cases differently.

Behavioral modeling in SysML v2

SysML v2 treats behavior as part of a more regular, text-first declarative language, aligned with the broader definition–usage discipline and an integrated expression language. Behavior is authored as explicit model elements (often textually) and then rendered as views when needed, which supports reviewability, reuse, and consistent semantics across representations.

- *Interactions* are modeled as first-class behavioral specifications whose ordering and message semantics can be expressed directly (in text) and visualized in sequence-style views. The emphasis shifts from combined-fragment diagram grammar to explicit model semantics that are easier to validate and diff.
- *Actions* follow the same definition–usage pattern as structure, and sequencing is expressed directly through explicit relations (such as successions, including *first … then …*, and explicit value-flow relationships). Because the expression language is integrated, guards, computations, and selection logic can be written as model expressions rather than relying on a large catalog of diagram node types.
- *State machines* are intentionally simplified relative to SysML v1: SysML v2 removes UML-style *pseudostates* from the language, producing a *dumbed-down* state-machine vocabulary with fewer special constructs. This reduces complexity and tool divergence, but it also means that patterns commonly expressed with pseudostates in v1 must be represented more explicitly, and often inelegantly, by

restructuring states and transitions and/or by expressing complex decision logic through actions and expressions rather than pseudostate nodes.

Textual notation as the primary representation: What it enables

SysML v2's textual notation is not a convenience feature; it is a primary language representation. That choice changes both authoring and lifecycle management.

Precision and reviewability

Text makes certain things *easier to do well*:

- You can define model elements with explicit scoping and naming discipline.
- You can review changes as diffs, not screenshots.
- You can apply consistent formatting and editorial standards (important for a tutorial text and for engineering teams).
- You can apply AI assistance to have an AI create textual representations of SysML models.

Versioning and merging

Diagram-first models are difficult to version in a way that supports meaningful merges. Text-first artifacts are naturally compatible with:

- Branching, merging, and code-review style practices
- Automated style checks and lint-like validation
- CI pipelines that run model checks (constraint evaluation, query-based regressions, etc.)

Even if a team never treats a model "exactly like code," SysML v2 makes that *possible* in a way SysML v1 never did reliably.

Automation and tool interoperability

SysML v2 is designed to participate in automation ecosystems. This includes a tool API defined in the language standard to specify a standard means for tool interoperability. The practical impacts include:

- The ability to query models using standard APIs rather than tool-specific export formats
- The possibility of scripted model transformations, analysis execution, and results injection as model attributes
- A clearer separation between the semantic model and any particular tool's diagram editor

This is one of the reasons SysML v2 feels "larger" than SysML v1: it is not just a language, but a language plus an ecosystem blueprint.

Diagrams vs. views: From a fixed diagram set to definable view mechanisms

One of the most noticeable differences between SysML v1 and SysML v2 is how model information is organized and presented. In SysML v1, modelers typically work with a familiar set of diagram types. SysML v2 takes a different approach, allowing views to be defined according to the information that needs to be presented. Understanding this distinction helps explain the shift in how modelers interact with and communicate their models.

SysML v1: Named diagram types as the organizing mental model

In SysML v1, modelers commonly learn the language through its diagram taxonomy:

- Requirements Diagram, Block Definition Diagram, Internal Block Diagram, Parametric Diagram, Activity Diagram, Sequence Diagram, State Machine Diagram, Use Case Diagram, Package Diagram (plus tool tables/matrices)

The diagram type often drives what the modeler thinks they *should* model next: "now I need an ibd," "now I need a par," and so on.

SysML v2: Views are model elements, and you can define your own

SysML v2 formalizes the idea that a diagram is a *view usage* of a *view definition* (and that viewpoint mechanisms can constrain what a view is intended to address). The shift is subtle but profound:

- A *view* is not merely a drawing canvas.
- A view is a model element that can *expose* some portion of the model, *filter* what it includes, and *render* it in a chosen format.
- "Diagram types" are not a fixed set you are trapped inside. SysML v2 provides standard view definitions, but it also supports user-defined view definitions that can be tailored to your stakeholders and domain.

This has two practical benefits:

- Views become purposeful artifacts. Instead of making a diagram because the methodology says so, you can define a view whose mission is explicit: it answers a question, supports a decision, or frames an argument.
- You can make the "missing diagrams" you want. If you want a requirements-style diagram or a domain-specific table/matrix, you can define it as a view definition and reuse it consistently (without waiting for a tool vendor to hardcode a new diagram type).

Mapping table: Key SysML v1 concepts to SysML v2 concepts

The following mapping is intentionally *not* a claim of equivalence. It is a practical translation guide, with notes about what changes in meaning or usage.

SysML v1 concept	**SysML v2 concept**	**What changed / practical note**
Block / Part property	part def / part item def / item	SysML v2 makes the definition–usage split explicit and pervasive. "Part" is a usage; "part def" is the reusable definition. In v1, if you needed a compound data or flow type, you used a block; in v2, you use an *item*
Value Type / Value Property	attribute def / attribute	Attributes are first-class; value typing and units are handled through libraries and explicit typing conventions.
Constraint / Constraint Property	constraint def / constraint	A constraint in v2 is a kind of calculation that returns a Boolean

SysML v1 concept	**SysML v2 concept**	**What changed / practical note**
constraint block, constraint property	calc def / calc usage	Rather than isolating quantitative logic into parametric diagrams, v2 supports calculations broadly across the model.
Activity / Action	action def / action	Actions follow the same definition–usage split; sequencing is expressed with successions (including specialized forms such as first … then …).
State Machine / State	state def / state	The same definitional discipline applies; v2 aims for more regular semantics and integration with occurrence concepts. States in v2 are special kinds of actions.
Association / Connector	connection def / connection	Connections are first-class and can carry attributes/ constraints; specialized forms (bind, succession) are syntactic sugar on a common underlying concept.
Interface	(no direct SysML v2 equivalent)	A SysML v1 *Interface* is a UML behavioral contract (services/ events) concept; SysML v2 does not provide a one-to-one counterpart as a standalone classifier-type contract element.
Interface Block	port def	A SysML v1 *Interface Block* (used to type proxy ports and capture the interaction content) maps most closely to a SysML v2 *port definition*, where directed features (for example, *in/out* items/attributes) define the allowable interaction at a port.
Connector	connection or interface	A SysML v1 connector maps to a SysML v2 *connection usage*; when the connector's semantics are meant to be made explicit as a contract governing the interconnection, SysML v2 additionally models that with an *interface* associated with the connection.
Proxy Port	port	A SysML v1 *proxy port* maps to a SysML v2 *port usage*; the port's type is typically a *port def* (corresponding to an Interface Block in v1), with conjugation used to ensure compatible directionality.
Item Flow	item definitions/usages and flow features on ports/connections	The flow vocabulary is integrated with items and directed features rather than being an annotation on connectors.
Operation	action	V1 had the notion that a block could have an operation whose implementation was defined in a method. V2 has no such concept. A part can have actions, and those actions can be invoked.

SysML v1 concept	SysML v2 concept	What changed / practical note
«satisfy», «verify»	satisfy statement; verify statement	v2 keeps trace intent but expands the modeling space around verification via explicit verification cases and verdict concepts.
«trace»	dependency	V2 does not represent trace as a relation; use dependency instead
Allocation	allocation	Allocation remains a first-class relation; in v2 it integrates naturally with definition–usage and can be shown in tailored views and tables.
View / Viewpoint (v1 concept)	view def / view and viewpoint def / viewpoint	V1 defined a fixed set of views: 9 diagrams, one requirements table, and one allocation matrix, although practically all SysML v1 tools allow the creation of custom tables and matrices. In v2, views are model elements with expose/filter/render semantics. You can define your own views and make them reusable assets.
Package	package namespace	Packages remain namespaces but are more central to configuration management and reuse. V2 includes explicit import rules and semantics.
Requirement	requirement def / requirement	Requirements are constraint-bearing elements with explicit subjects, assumptions, and required constraints; they can be more directly executable.
test case	verification case	Verification is standardized as a case specialization, enabling clearer modeling of evidence and verdict.
Stereotype	metadata def / metadata usage	SysML v1 extends UML using stereotypes and tagged values; SysML v2 uses *metadata definitions* as first-class language elements and applies them directly to model elements to capture classification, status, risk, review information, and so on, without relying on UML profiling.

Table B.1: Key SysML v1 concepts mapped to SysML v2 concepts

Mapping table: SysML v1 diagram types to SysML v2 views and representations

SysML v2 does not simply rename the nine (or commonly taught) SysML v1 diagram types. Instead, it provides standard view definitions and mechanisms to create additional view definitions. The following mapping is therefore the closest conceptual counterpart, not a strict one-to-one.

SysML v1 diagram	Typical intent in v1	SysML v2 counterpart	Notes
Requirements Diagram (req)	Requirements, derivations, satisfy/verify, traceability	General view + requirements-focused filter; grid/table views	In v2 you can define a requirements-focused view definition; requirements tables/matrices naturally fit grid views.
Block Definition Diagram (bdd)	Type structure, generalization, associations	General view; browser/tree renderings	Often best represented as a general view rendered as a tree (taxonomy).
Internal Block Diagram (ibd)	Internal composition, ports, connectors, flows	Interconnection view	Interconnection views are oriented toward "what connects in this context."
Parametric Diagram (par)	Constraints among properties and binding	General view interconnection view	Many parametric-style concerns can be shown as constraint/calc elements and rendered as appropriate views.
Activity Diagram (act)	Flow of actions, control/data flows	Action flow view	v2's action modeling and successions support direct depiction of action sequencing.
Sequence Diagram (sd)	Message-based interaction over time	Sequence view	v2 can represent interaction scenarios and render them as sequence-style views.
State Machine Diagram (stm)	States and transitions	State transition view	State modeling can be rendered in state-transition views.
Use Case Diagram (uc)	Actors and use cases	General view	Use cases are specialized cases; case views can render these relationships.
Package Diagram (pkg)	Package containment, dependencies	General view; browser/tree views	Package structure is often better shown as a tree or general view with membership relations.
Tables / Matrices (tool features)	Allocation matrix, requirements tables, trace matrices	Grid view (tabular/matrix rendering)	In v2, tabular renderings are part of the view mechanism, not just a tool add-on.

Table B.2: SysML v1 diagram types mapped to SysML v2 views and notation

Practical migration guidance: How to avoid "v1 thinking traps"

- **Stop asking "Which diagram do I draw next?"**
  - In SysML v2, the better question is: *What concern am I addressing, and what view best exposes the model evidence for that concern?* Views are purposeful, and you can define the ones you need.
- **Treat requirements as structured constraints, not decorated text.**
  - You still write good doc text, but you also model *subjects*, *assumptions*, and *required constraints* so the requirement is precise and evaluable.
- **Use cases are not the only "case" worth modeling.**
  - Analysis and verification are not second-class citizens. Model them explicitly as cases so the model captures not just *what is required*, but also *how you evaluate and demonstrate* it.
- **Learn the definition–usage split early and apply it consistently.**
  - If you internalize this pattern, most of SysML v2 becomes easier to read. If you resist it, SysML v2 will feel unnecessarily complex. Understand that v2 is not completely regular. The def/usage paradigm does not apply to packages, namespace, quantity kinds, units, or many other elements. However, v2 is still far more regular than v1.
- **Exploit text-first benefits without abandoning diagrams.**
  - Use text for precision and versioning; use views for communication and stakeholder reasoning. SysML v2 is strongest when you treat these as complementary, not competing.

References for SysML v1

- SysML v1 specification overview and requirements-diagram intent (including satisfy/verify relationships). (`omg.org`)
- SysML v1.6 "About" page (official OMG spec context). (`omg.org`)
- Cameo/No Magic SysML diagram descriptions (practical descriptions of common v1 diagram types). (`docs.nomagic.com`)
- SysML v1 requirement diagram description (tool documentation reflecting the standard intent). (`docs.nomagic.com`)

C

Exercises

Chapter 2

Here are some exercises to allow you to practice using the Cameo tool and gain proficiency with it.

Exercise 2.1: Navigating the containment tree and text view

Estimated Time: 10–15 minutes

Goal

Develop confidence navigating the model hierarchy in Cameo and understanding how the text view and **containment tree** represent the same underlying model.

Tasks

1. Open a simple model and add two requirements, three part usages, and one part definition to the existing package with the **Create Element** menu in the **containment view**.
2. Add a new namespace and name the package it owns to `SamplePkg`.
3. In the **containment tree**, expand and collapse namespaces and locate specific elements by name.
4. Select an element and view its definition in the text view (right-click the element and select **Textual View > Open**) observing how the selection in one view highlights the corresponding element in the other.
5. Modify the documentation (doc) of an element in the text view and observe the change reflected in the browser.
6. Rename a model element in the **containment tree** and confirm that its text definition updates accordingly.

Exercise 2.2: Creating and managing views (diagrams)

Estimated Time: 10–12 minutes

Goal

Practice creating diagrams (views), adding symbols, and understanding how symbols relate to the model elements they represent.

Tasks

1. Create a new package named `DemoPkg`.
2. Inside it, create a General View using the same approach described in the *Working in the General View: a quick tutorial* section.
3. Add three model elements (such as `part def A`, `part def B`, `part def C`) in the text view.
4. Return to your diagram and drag those model elements from the containment tree onto the view to create symbols.
5. Add the parts A and B to the part def C with the **Feature Membership** relation under the **Other Relations** category in the diagram palette.
6. Delete a model element from the containment tree and observe that all symbols representing it are removed from diagrams automatically.

Exercise 2.3: Using the Specification window and element properties

Estimated Time: 10–15 minutes

Goal

Gain hands-on practice inspecting and editing model element properties using the **Specification** window.

Tasks

1. Create a simple element such as `part def Sensor`.
2. Open the **Specification** window for the element.
3. Locate and edit properties such as:
    - name
    - documentation
    - visibility
    - **Default value** of an attribute (add one if needed)
4. Change a semantic property (e.g., default value) and confirm that the text view updates.
5. Change a *symbol-only* property (such as color or shape) through the **Symbol Style Dialog** and confirm that the underlying text definition remains unchanged.

Exercise 2.4: Working with palettes and quick-add features

Estimated Time: 10–12 minutes

Goal

Practice using Cameo's diagram palette, quick-create tools, and auto-layout to accelerate diagram creation.

Tasks

1. Open or create a new **Interconnection** or **General View**.

2. Using the **palette**:
    - Create three new elements directly on the diagram
    - Create at least one relationship (such as a connection or dependency)
3. Use the quick-create ("**smart link**") tool to add another element by dragging a relationship from one symbol into empty space.
4. Apply an **auto-layout** option to the diagram and observe how symbols are rearranged.
5. Move symbols around and experiment with grouping or alignment features.

Exercise 2.5: Filtering, searching, and showing/hiding elements in views

Estimated Time: 10–15 minutes

Goal

Become comfortable filtering model content to create meaningful, uncluttered views.

Tasks

1. Open a view containing at least several model elements (use previous exercises if necessary)
2. Use the **Quick Find** function to locate elements by partial name.
3. Apply a visibility filter to hide selected types of elements (such as attributes or certain relationship types)
4. Hide one symbol manually using **Hide** and then reveal it using **Show Elements in Diagram**.
5. Create a second view of the same package and experiment with different filters or visibility settings to compare how each view exposes different aspects of the same model.

Chapter 4

The ideas in this chapter matter only if they can be applied in a real model. The following exercises are intended to move the discussion from principle to practice by having you define, refine, and evaluate requirements in SysML v2. As you work through them, focus not just on syntax, but on the underlying discipline of writing requirements that are precise, testable, and properly related to the rest of the system model.

Exercise 4.1: Create simple requirements

Create a new model containing the requirements and relations shown in *Figure 4.15*.

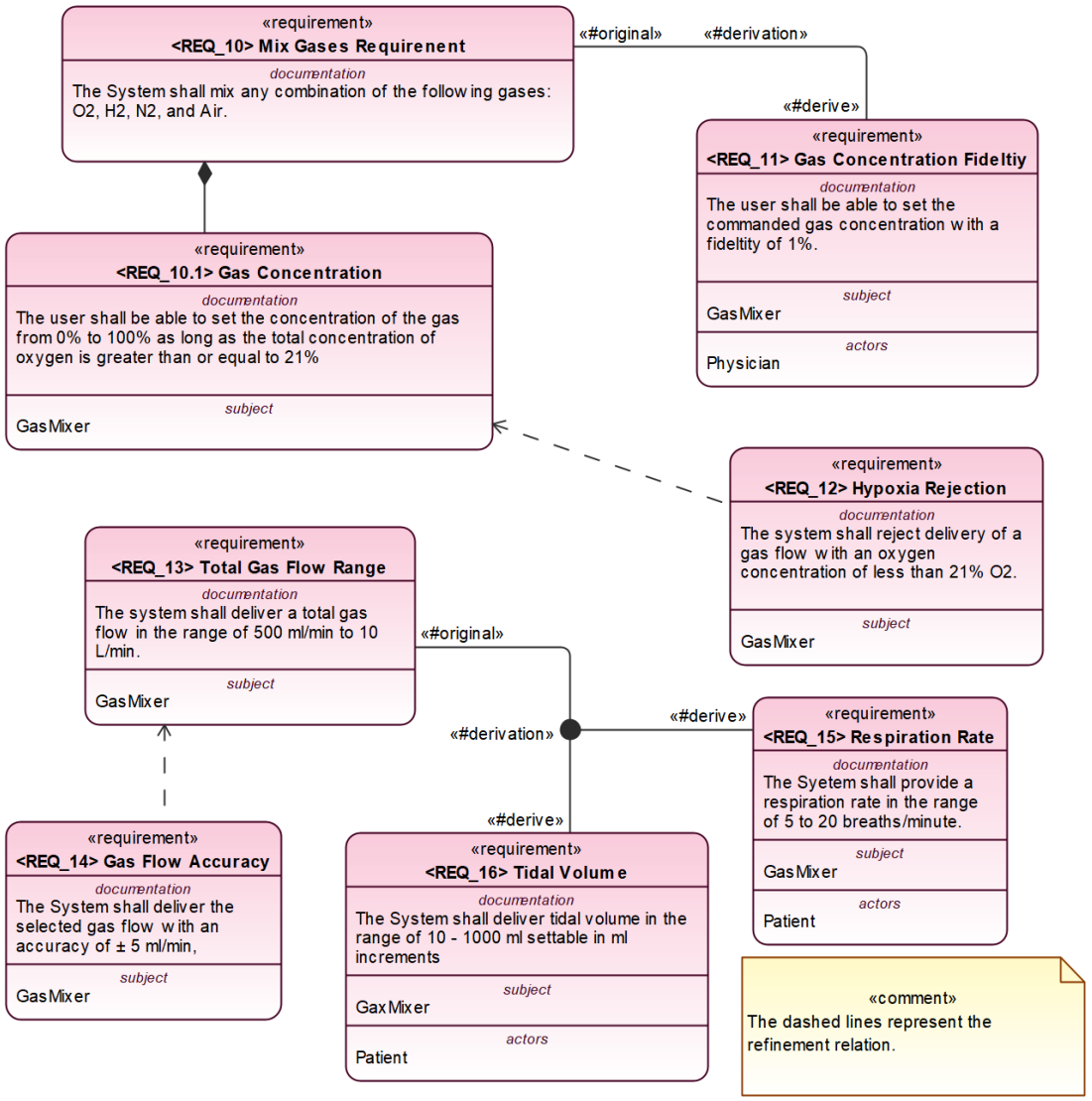


Figure 4.15– View for Exercise 4.1

Exercise 4.2: Creating a requirements table

To the model created in *Exercise 4.1*, add a requirements table that exposes the requirement name, subject, documentation, derived from, derived to, and refined from columns.

Exercise 4.3: Adding constraints

To the model created in *Exercise 4.1*, add the following using constraints, parameters, and attributes as necessary:

- `Mix Gases Requirement`: Add an `assume` constraint that the inputs have at least one oxygen or one air gas supply connected.
- `Gas Concentration` **requirement**: Add a `require` constraint that formalizes its documentation text.
- `Gas Concentration Fidelity` **requirement**: Add a `require` constraint that formalizes its documentation text.
- `Hypoxia Rejection` **Requirement**: Add a `require` constraint that formalizes its documentation text.
- Add a part definition `GasMixer` that defines a part `gasMixer` that connects to multiple (*) gas supplies. Add satisfy relations from the `gasMixer` to each of the requirements.
- Add a **verification case** `TidalVolumeTestCase` that verifies the `Tidal Volume` requirement, and another verification case `RespirationRate` that verifies the `Respiration Rate` requirement

Exercise 4.4: Rewriting requirements

For the following requirements, identify what rule(s) they violate from *Table 4.3*, rewrite the requirements to be better, and then create a set of requirements for these rewritten statements, visualized both diagrammatically and in a table form.

Rewrite each of the requirements so that it is well-formed:

- The system shall display the state of the elevator
- The temperature of the boiler should not get too high
- The computation shall be delivered immediately after the user presses the **submit** button
- The bin should not deliver too many beans
- The water shall not be too hot, too cold, or at too high a pressure
- Temperature shall be measured every 1.5 seconds
- Lower Power Mode shall use minimum power
- If possible, the system will alert the user prior to an impending collision

Exercise 4.5 Microwave oven requirements

You are responsible for specifying the requirements for the next microwave oven. Create the following:

- Create at least eight stakeholder concerns
- Create a model with `>=20` system requirements, of which
- At least 10 must be functional

- At least 5 must be quality of service
- There must be > 5 relations among the requirements
- Use canonical form and guidelines for requirement documentation (such as testability)
- Add formal constraints to refine the textual requirements
- Relate concerns to appropriate requirements
- Create one or more requirements diagrams visualizing the requirements and their relations
- Create a requirements table that exposes the requirements and their textual specifications

Exercise 4.6: Adding derivations and refinements to the gas mixer model

Starting from the model created in *Exercise 4.3* (gas mixer):

- Create a top-level requirement, `Maintain Safe Respiratory Gas Mix`, that informally states that the delivered gas mixture shall remain within safe oxygen and carbon dioxide bounds for the patient
- Make the existing requirements (`Mix Gases Requirement`, `Gas Concentration`, `Gas Concentration Fidelity`, `Hypoxia Rejection`) derived from this top-level requirement. Use explicit derivation relations (`binary` or *n*-way) rather than simple «`trace`»`dependencies`.
- For `Gas Concentration` and `Hypoxia Rejection`, add refinement relations to show that their formal constraints refine the informal documentation text of `Maintain Safe Respiratory Gas Mix`.
- Update the requirements diagram and requirements table to expose *derived from*, *derived to*, and *refined from* columns, and verify that all four lower-level requirements are connected back to the top-level requirement

Exercise 4.7: Modeling satisfaction by design elements

Continuing with the gas mixer model:

- Create a system design structure with at least the following part definitions: `GasMixerController`, `GasSensorArray`, and `GasValveAssembly`.
- Add `satisfy` relations from these parts to the requirements as follows:
- `GasMixerController` satisfies `Gas Concentration Fidelity` and `Tidal Volume` requirements
- `GasSensorArray` satisfies `Gas Concentration` and `Hypoxia Rejection` requirements
- `GasValveAssembly` satisfies `Mix Gases Requirement`
- Where appropriate, use reference subsetting (for example, assign specific parameter values or limits in the satisfying parts that correspond to attributes defined in the requirements)

Exercise 4.8: Adding verification cases and linking them to requirements

Still using the gas mixer / ventilator context:

- Define at least four verification cases, for example:
  - `HypoxiaRejectionTestCase`
  - `GasConcentrationAccuracyTestCase`
  - `GasConcentrationFidelityTestCase`
  - `MixGasesFunctionalTestCase`
- For each verification case, specify:
  - Its subject (for example, `GasMixer`, `GasMixerController`, or the overall ventilator system)
  - A brief documentation statement describing the test or analysis to be performed
  - Add `verify` relations from each verification case to the requirement(s) it covers.
- Ensure that.
  - Each of the primary gas-related requirements has at least one verification case
  - At least one verification case verifies more than one requirement (for example, a test that simultaneously checks concentration and hypoxia rejection)
- Create a Verification Coverage table showing, for each requirement, which verification cases are linked to it. Identify any requirements that currently have no verification coverage.

Exercise 4.9: Microwave oven — derivation, refinement, satisfaction, and verification

Extend the model from *Exercise 4.5* (microwave oven requirements):

- Select at least three high-level stakeholder requirements (for example, `Cook Food Evenly`, `Ensure User Safety`, `Minimize Energy Consumption`).
- For each selected stakeholder requirement, create two or more derived system requirements (for example, door interlock behavior, power-level control, turntable rotation, over-temperature protection). Use explicit derivation relations to connect each derived requirement back to its parent stakeholder requirement.
- Choose at least two system requirements and:
  - Add formal constraints that refine their documentation (for example, temperature limits, maximum allowed leakage power, timing bounds)
  - Add explicit refinement relations to show that these constraints refine the textual statement
- Introduce at least three design parts (for example, `DoorInterlock`, `MagnetronControl`, `UserInterfacePanel`) and add `satisfy` relations from these parts to the appropriate system requirements. Where feasible, use reference subsetting to assign design values that come from requirement attributes.

- Define at least three verification cases (for example, `DoorInterlockSafetyTest`, `EvenHeatingTest`, `PowerLevelAccuracyTest`) and connect them to the requirements with `verify` relations.
- Update your requirements diagram(s), requirements table(s), and concern–requirement matrix to reflect the new derivations, refinements, satisfactions, and verifications

Chapter 5

Exercise 5.1: Use case identification for AED system

Estimated time: 20–30 minutes

Create a use case view from a provided list of system elements.

The **automated external defibrillator** (**AED**) is a portable device for delivering an electrical shock to reset the heart in cases of ventricular fibrillation or heart failure. Using SysML v2, construct a use case view for an `AED` with the following system elements:

- Actors: `Bystander`, `Medical Technician`, `Patient`.
- Create the following use cases:
  - `DeliverShock`
  - `AssessPatient`
  - `ProvideInstructions`
  - `RunSelfDiagnostics`
  - `LogEventData`
- Model each use case as a **case usage**, with an identified objective, subject, and appropriate actor(s)
- Select one use case and create at least 5 requirements. Relate the use case and the requirements with relationships.
- Create a diagrammatic view of the set of use cases
- Create a diagrammatic view of the selected use case with its related requirements

Exercise 5.2: Use case discovery for target designation system

Estimated time: 20–30 minutes

Discover and model use cases from a short system description.

A target designation system for an air-to-ground strike platform detects, tracks, and confirms targets using onboard and external sensors. It communicates with a command center and fire control unit. Complete the following tasks:

- Identify at least five use cases, three actors, and a single subject
- For each use case, identify its subject, actors, and objective
- Define the relationships between actors, use cases, and the subject
- Add five requirements for each use case, using dependency relations to relate them to the appropriate use case
- Create a use case view in graphical SysML v2

Exercise 5.3: Action modeling of the InfuseDrug use case

Estimated time: 30–40 minutes

Refine a use case using an action with nested actions detailing the sequence of system functions necessary to provide the capability. Model the behavior of an `InfuseDrug` use case in a medical `infusion pump` system:

- Define the `InfuseDrug` use case with an actor, subject, and objective
- Create an action with nested actions (representing system functions) to define how the use case behavior unfolds over time:
  - `Nurse selects dosage`
  - `Pump verifies prescription`
  - `System checks for prior infusions`
  - `Drug infusion starts`
  - `Confirmation is logged`
  - `Infusion continues until the nurse disables the pump`
- Include at least five requirements and create dependency relationships from the use case

Exercise 5.4: State machine for auto-land on signal loss

Estimated time: 30–45 minutes

Refine the `AutoLandOnSignalLoss` use case using a state machine.

An **unmanned air vehicle** (**UAV**) can fly under the control of a remote controller operated by a UAV operator. The UAV can receive commands from the controller to change orientation (yaw), adjust altitude, and fly. This requires constant communication with the handheld controller. To ensure that it remains connected, if no commands are received from the controller, the UAV will issue a **life-tick response** within 10 s and expect a reply from the controller. If one is not received, the UAV will immediately perform a controlled landing until communication can be reestablished.

These are the tasks to be performed:

- Define the use case `AutoLandOnSignalLoss` specifying the actors, subject, and objective
- Create a state machine for the use case that models at least the following states:
  - `Off`
  - `Idle`
  - `Normal flight`
  - `Auto-Landing`
- Add item usages to represent the events of interest to the UAV for the state machine
- Add transitions triggered by appropriate events, such as the reception of an item from the controller, a timeout, or an internally generated event
- Define at least five requirements and model dependency relations from the use case
- Add appropriate pre- and postconditions and invariant constraints

Exercise 5.5: Activity flow for the EngageTarget use case

Estimated time: 45–60 minutes

Model a detailed activity flow for a tactical `EngageTarget` use case.

The **fire control system** (**FCS**) supports a basic Engage Target capability used by a `FireControlOperator` under the direction of an external `CommandSystem`. The `FCS` receives an engagement order and operator input, then acquires the designated target using available sensor data. Once a valid target is detected, the `FCS` calculates a simple engagement plan that identifies the weapon to use and the predicted enemy position. The `CommandSystem` must provide firing authorization before the target can be engaged.

If authorized, the FCS commands the selected weapon to launch. After the target is engaged (that is, hit by the missile), the system performs a simple assessment, checking sensor indications or operator confirmation to determine whether the target was destroyed, and this information is passed on to both the `FireControlOperator` and the `CommandSystem`.

These are the tasks to be performed:

- Define actors, subject, and objective
- Add `in` and `out` parameters for items and data exchanged with the actors
- Specify the action with nested actions and succession relations between them. Include actions such as the following:
  - `Acquire target`
  - `Determine combat ID` (unknown, friend, foe, neutral)
  - `Calculate engagement plan`
  - `Authorize fire`
  - `Launch weapon`
  - `Assess battle damage`
- Include decision, merge, fork, or join nodes as appropriate
- Add appropriate pre- and postconditions and invariant constraints
- Model at least 10 requirements and their dependency links from the use case

Exercise 5.6: VentilatePatient use case with state machine

Estimated time: 45–60 minutes

Use a state machine to model lifecycle control of the `VentilatePatient` use case.

The `Ventilator` provides a **ventilatory support** capability that assists a clinician in maintaining a patient's breathing. The system operates under the supervision of the `Clinician`, who initiates, adjusts, and terminates ventilation as needed. Once ventilation is started, the system transitions through a small number of operating modes, such as initializing airflow, delivering breaths, monitoring patient response, and pausing or stopping ventilation when commanded.

Throughout operation, the system continually monitors sensor data, such as airway pressure or **tidal volume** (the volume per breath), and may transition into safety or alarm states if abnormal conditions (no input gas flow, high pressure, low pressure) are detected.

Complete the following tasks:

- Construct the `VentilatePatient` use case with an appropriate subject, actors, and objective
- States:
  - `Off`
  - `Initializing` (setting up operational parameters such as ventilation rate and tidal volume)
  - `Ventilating`
    - `Inhaling`
    - `Exhaling`
    - `Intra-breath pause`
  - `Error state`
- Trigger transitions by events such as threshold breaches, timeouts, or user inputs
- Add appropriate pre- and postconditions and invariant constraints
- Include at least 10 requirements for the use case and link them via dependency

Exercise 5.7: Safe mode transition for satellite system

Estimated time: 45–60 minutes

Use a state machine to refine the `EnterSafeMode` use case.

The `Satellite` provides an `EnterSafeMode` capability that protects the spacecraft when serious onboard anomalies occur. During normal operation, the `Satellite` remains in the `NominalOperation` state, performing routine mission and payload activities while the `GroundStation` supervises health and status. If the satellite detects an anomaly, such as a subsystem malfunction, a sensor discrepancy, or a power-level warning, it transitions into `FaultDetected`, where it performs quick diagnostic checks to classify the fault and determine whether it is recoverable.

Within `FaultDetected`, the satellite evaluates a small set of internal conditions to decide the next state. If the fault is minor or recoverable, the satellite will retry standard operations. However, if the fault is classified as critical, for example, if more than five recovery attempts fail or a critical error is discovered, the satellite triggers a transition to `EnteringSafeMode`.

In the `EnteringSafeMode` state, the satellite performs a controlled sequence of internal actions designed to protect the spacecraft: disabling nonessential payloads, reorienting the solar panels for maximum power and thermal stability, and switching communications into a robust low-rate mode. If these actions complete successfully, the spacecraft moves to `SafeModeStable`, where it maintains a safe configuration, reports its condition to the ground station, and waits for further instructions (such as a `ShutDown` or a `Restart` command from the `GroundStation`).

These are the tasks to be performed:

- Create the subject part def, the use case, and its actors, subject, and objective
- Identify the states and modes for the use case from the problem description
- Specify the transitions among the states, including their triggering events and guards, if any
- Define internal actions (such as disable payload, reorient panels) and add them to the states (as entry, exit, or do actions) or to the transitions
- Add pre- and postconditions and invariants as constraints
- Include at least 10 requirements and connect them to the use case

Chapter 6

The following exercises are designed to help reinforce the foundational modeling concepts introduced in this chapter. They offer a progression from basic structural modeling to more complete system specifications that incorporate attributes, parts, items, flows, parameters, behaviors, and occurrences. Each exercise is grounded in a real-time or embedded system scenario, giving readers a practical and domain-relevant context in which to apply what they have learned. By working through these examples, readers will gain hands-on experience creating SysML v2 models that are both syntactically valid and semantically meaningful.

Some exercises intentionally integrate concepts that are introduced in later chapters, such as calculations, actions, and state machines. If you have not yet studied those topics, you may either complete the structural and informational portions now and return to the remaining tasks later, or defer the exercise until after reading the relevant chapter.

Exercise 6.1: Heart rate monitor system context

Objective: Define a simple system context using parts, attributes, and composition.

Estimated time: 20–30 minutes

System description: A wearable heart-rate monitor consists of a sensing component that measures heart rate and a small display that presents computed values to the user. The system maintains internal configuration parameters such as thresholds, sampling rates, and units for display. Modelers must construct the basic structural elements and identify the information each part maintains to support monitoring behavior.

Tasks:

1. Define a `part def` for `HeartRateMonitor`
2. Add part usages for a `Sensor` and a `Display` as components of the monitor.
3. Add at least three attributes to each part definition (such as `samplingRate`, `displayUnits`, `threshold`)
4. Use `doc` comments to explain the purpose of each part and attribute.

Concepts practiced: `part def`, attribute, composition, basic structural modeling

Exercise 6.2: Modeling flows in a missile warning system

Objective: Use item definitions and flows to represent information moving between system components.

Estimated time: 25–35 minutes

System description: A missile warning system contains a distributed sensor array that detects potential threats and forwards detection messages to a processing unit that evaluates and classifies them. The system relies on well-defined information flows that carry detection metadata such as signal strength and time of observation.

Tasks:

1. Define an `item def` for `ThreatSignal`, including attributes such as `threatType : String`, `signalStrength : Real`, and `timestamp : DateTime`.
2. Define a `part def` for `MissileWarningSystem` containing `SensorArray` and `WarningProcessor` subparts.

3. Create a flow of `ThreatSignal` from the `SensorArray` to the `WarningProcessor`.
4. Add `doc` comments explaining the meaning, direction, and role of the flow.

Concepts practiced: `item def`, flow, input/output features, modeling signal-based communication

Exercise 6.3: Parameters and calculation in an autonomous braking module

(This exercise uses actions and calculations, which are covered later in the book. If those concepts are not yet familiar, complete the structural and parameter portions now and return to the action and calculation tasks after reading the relevant chapters.)

Objective: Introduce behavioral modeling with parameters and simple computation

Estimated time: 30–40 minutes

System description: An autonomous braking module receives sensor information about the vehicle's speed and the distance to obstacles ahead. It computes an appropriate braking force to avoid or mitigate collisions. This exercise requires defining the structural element and a calculation that derives a braking command from input parameters.

Tasks:

1. Define a `part def` for `BrakingModule`.
2. Create an `action def` (such as `AutonomousBraking`) with parameters such as `currentSpeed`, `obstacleDistance`, and `brakeForce` (all `Real`)
3. Define a calculation that computes `brakeForce` based on the other parameters.
4. Document the rationale behind the chosen parameters and relationships using doc comments.

Concepts practiced: `behavior def`, parameter modeling, calculation or constraint definition, expressing system logic

Exercise 6.4: Occurrences and state for satellite solar-panel deployment

(This exercise uses states, transitions, and events, which are covered later in the book. If those concepts are not yet familiar, complete the structural and occurrence-related portions now and return to the state-machine tasks after reading the relevant chapters.)

Objective: Explore state modeling and basic occurrence semantics

Estimated time: 35–45 minutes

System description:

A satellite deploys its solar panels shortly after reaching orbit. Deployment follows a prescribed sequence and depends on events sent from onboard controllers. The system transitions through structural and behavioral occurrences as deployment proceeds.

Tasks:

1. Define a `part def` for `Satellite` with a nested `SolarPanelAssembly` part.
2. Create a `state def` (such as `DeploymentBehavior`) with states `Stowed`, `Deploying`, and `Deployed`
3. Define items to represent events such as `initiateDeployment` and `deploymentComplete`.
4. Specify transitions between states, including entry logic and conditions.
5. Add `doc` comments describing the purpose of each state and when transitions occur.

Concepts practiced: `state def`, **event occurrence**, **transition semantics**, structural and behavioral occurrences

Exercise 6.5: Integrated modeling: infusion pump controller

(This exercise intentionally integrates structural and behavioral concepts. If action modeling is not yet familiar, complete the structural, item, flow, and parameter portions now and return to the action-related task after reading the relevant chapter.)

Objective: Combine structural, informational, and behavioral modeling concepts into a unified example

Estimated time: 45–60 minutes

System description: An infusion pump controls and monitors the delivery of medication to a patient. A flow sensor reports the measured flow rate to a controller, which compares it to a desired rate and raises alerts or adjusts output as needed. This exercise brings together several structural and behavioral modeling elements.

Tasks:

1. Define a `part def` for `InfusionPump` with nested `Controller` and `FlowSensor` parts.
2. Add attributes `desiredRate` and `actualRate` to the `Controller` part.
3. Define an `item def` for `DrugFlow` containing at least `rate : Real` and `drug : String` attributes.
4. Model a flow of `DrugFlow` from the `FlowSensor` to the `Controller`.
5. Create an action with parameters and constraints ensuring `actualRate` remains within 5% of `desiredRate`.
6. Use `doc` comments to explain how parts, items, parameters, and constraints interact to maintain safe operation.

Concepts practiced: integrated structural modeling, item and flow modeling, parameterization, constraint-based behavioral verification.

Chapter 7

The following exercises are designed to help you apply and internalize the concepts presented in this chapter on data and information. Each task focuses on a specific aspect of data modeling, including the use of quantity kinds, units, attribute and item structuring, metadata, and reusable schemas. The exercises progress in complexity and scope, giving you opportunities to practice both foundational skills and more nuanced modeling decisions. By completing these activities, you will reinforce your understanding of how data and information are represented in SysML v2 and gain experience structuring models that are semantically rich, analyzable, and clearly communicative.

Exercise 7.1: Basic attribute typing with quantities and units

Objective: Practice defining attributes using correct quantity kinds and units from standard libraries.

Estimated time: ~10 minutes

System description:

A `WaterTank` is a simple physical storage vessel used in residential and industrial systems. Its operational characteristics depend on measurable quantities such as its current water volume, its maximum capacity, and its temperature. Properly typing these values ensures physically meaningful modeling and prevents invalid states in simulations or analyses.

Tasks:

1. Define a `WaterTank` part.
2. Add three attributes:
    - `currentVolume` typed with a quantity kind for volume (such as `VolumeValue`) and expressed in liters
    - `maximumCapacity` using the same quantity kind and unit
    - `temperature` typed using a temperature quantity kind (such as `ThermodynamicTemperatureValue`) and expressed in degrees Celsius
3. Assign reasonable default values to each attribute.
4. Ensure that each attribute uses the appropriate `ISQ` quantity kind and `SI` unit.

Concepts practiced: Attributes, quantity kinds, units, default values, `ISQ`/`SI` library usage, physical consistency.

Exercise 7.2: Creating and using custom quantities and units

Objective: Define a custom domain-specific quantity kind and a derived unit, then apply them in a part definition.

Estimated time: ~15 minutes

System description:

Fermentation systems in brewing and bioprocessing release CO_2 at rates that must be monitored for safety and product quality. Modeling this requires a domain-specific measurement that does not exist in the `ISQ` library. By

defining a new quantity kind and derived unit, the model can represent CO_2 production accurately and consistently across analyses.

Tasks:

1. Define a new quantity kind `FermentationRateValue`, extending an appropriate scalar quantity value type.
2. Define a derived unit `gCO2_per_hour` representing grams of CO_2 produced per hour.
3. Define a `Fermenter` part.
4. Add an attribute `rateOfCO2Production` typed by the new quantity kind.
5. Assign a default value of `23.5[gCO2_per_hour]`.

Concepts practiced: User-defined quantity kinds, custom-derived units, domain-specific modeling, typing attributes with custom measurements.

Exercise 7.3: Building a data schema with items and attributes

Objective: Create a reusable item-based data schema and incorporate it into a system part.

Estimated time: ~20 minutes

System description:

Autonomous floor-cleaning robots track battery status so they can schedule recharging and avoid shutdown during operation. This information includes charge levels, charging state, voltage, and battery temperature, forming a coherent data object that should be modeled as a structured item for reuse and clarity.

Tasks:

1. Define an item `BatteryStatus` containing the following:
    - `chargeLevel` as a percentage value
    - `isCharging` as a Boolean
    - `voltage` typed using an electrical potential quantity kind and expressed in volts
    - `temperature` expressed in degrees Celsius
2. Define a `RobotVacuum` part.
3. Add an item `battery` typed by `BatteryStatus`.
4. Assign reasonable defaults to all inner attributes.

Concepts practiced: Item definitions, structured data schemas, typed attributes, default values, reusable information objects.

Exercise 7.4: Using metadata to add semantic annotations

Objective: Apply metadata using both inline (`@`) and reference-based (`#`) annotations.

Estimated time: ~25 minutes

System description:

A weighing scale measures object mass but requires periodic calibration to guarantee accuracy. Capturing calibration metadata allows the system model to store information about who calibrated the device, when the calibration occurred, and what accuracy level is expected. This enhances traceability and quality assurance in measurement-driven systems.

Tasks:

1. Define metadata `CalibrationMetadata` with properties `calibratedBy`, `calibrationDate`, and `accuracy`.
2. Create a named metadata instance `StandardCalibration` with typical values representing standard laboratory calibration.
3. Define a `WeighingScale` part.
4. Add an attribute `massReading` typed using an appropriate `ISQ` mass quantity kind and unit (such as kilograms).
5. Add an attribute `tareWeight` typed the same way.
6. Apply `CalibrationMetadata` to `massReading` using inline (`@`) metadata with custom values.
7. Apply the metadata to `tareWeight` using the reference form (`#StandardCalibration`).

Concepts practiced: Metadata definitions, inline and referenced metadata application, annotation of measurements, traceability, `ISQ`/`SI` typing.

Exercise 7.5: Modeling an information schema for a digital thermostat

Objective: Integrate items, enumerations, attributes, units, and metadata into a coherent information model.

Estimated time: ~30 minutes

System description:

A digital thermostat processes configuration settings and environmental sensor readings to regulate indoor climate. Both sets of information (user settings and sensor data) form structured entities with constraints, physical semantics, and metadata annotations. Modeling these information structures clarifies system behavior, interfaces, and validation rules.

Tasks:

1. Define an item `ThermostatSettings` with the following attributes:
    - `targetTemperature` typed with a temperature quantity kind and appropriate unit
    - `mode` typed by an enumeration with literals `Off`, `Heat`, `Cool`, `Auto`
    - `isLocked` as a Boolean
2. Define an item `SensorReadings` with the following attributes:
    - `roomTemperature` (temperature quantity kind + unit)
    - `humidity` using a derived unit such as %RH
    - `batteryLevel` as a percentage

3. Apply `QuantifiedValue` metadata inline to both `roomTemperature` and `humidity`.
4. Define a part `ThermostatUnit` containing item usages typed by these two item definitions.
5. Provide reasonable default values for all attributes.

Concepts practiced: Item-based schemas, enumerations, metadata, use of derived units, information modeling for cyber-physical systems.

Chapter 8

This chapter introduced *part definitions* and *part features*, including attributes, items, flows, ports, and multiplicity. The following exercises provide self-contained modeling challenges that focus on how *part features* define structure and behavior. More elaborate, system-level, and relational exercises appear in the next chapter after part relations have been discussed more thoroughly.

Exercise 8.1: Stereo amplifier

Objective: Model a structured audio subsystem using ports, items, flows, attributes, actions, and internal connections to represent signal processing and amplification.

Estimated time: ~30 minutes

Problem description: A stereo amplifier combines a `Preamp` and `PowerAmp` to process and amplify audio signals. The preamp conditions a low-level incoming signal and applies user-controlled gain, while the power amplifier boosts this processed signal to drive left and right speakers. The system receives an audio input and gain control signal at its boundary ports, processes and amplifies the signal internally, and outputs stereo signals. This exercise reinforces how item flows, attributes, port definitions, conjugation, and internal actions create a coherent structural and behavioral model.

Tasks:

1. Define `AudioSignal` as an item with attributes for amplitude and frequency.
2. Define `iAudioSignal` as a port with `in` and `out` item features of type `AudioSignal`.
3. Define `iGainCtrl` as a port with an `in` attribute `gainSetting: Real`.
4. Create `Preamp` with attributes (`volumeLevel`, `gain`), ports (`pInput: iAudioSignal`, `pOutput : ~iAudioSignal`), and an action (`adjustSignal`).
5. Create `PowerAmp` with attributes (`gain`, `outputPower`), ports (`pInput: iAudioSignal`, `pOutput : ~iAudioSignal`, `pGain : iGainCtrl`), an action (`amplify`), and a constraint linking `gain` and `outputPower`.
6. Define `StereoAmplifier` with parts (`preamp: Preamp` and `amp : PowerAmp`) and boundary ports (`pAudioIn`, `pGainControl`, `pLeftOut`, and `pRightOut`):
7. Create the following connections: `pAudioIn` → `preamp.pInput`; `preamp.pOutput` → `amp.pInput`; `pGainControl` → `amp.pGain`; `amp.pOutput` → `pLeftOut` and `pRightOut`
8. Demonstrate encapsulation by exposing only the boundary ports and hiding the internal parts.

Exercise 8.2: Greenhouse sensor node

Objective: Model an environmental sensor node using item flows, internal parts, actions, constraints, and a network-level aggregation.

Estimated time: ~25 minutes

Problem description: A greenhouse monitoring system uses distributed sensor nodes that each collect temperature and humidity readings at a specified sampling rate. Each node has its own sensors and a controller that packages data into an `EnvReading` item. The nodes export their readings to a central collector in a `SensorNetwork`. This exercise illustrates internal composition of multiple sensing parts, item flow modeling, and constraints on attribute values.

Tasks:

1. Define `EnvReading` as an item with attributes `temperature` and `humidity`.
2. Create `TempSensor`, `HumiditySensor`, and `NodeController` part definitions.
3. Create `SensorNode` with attributes `samplingRate` and `nodeID`; include parts `temp`, `humid`, and `controller`.
4. Add an action `collectData` producing an `EnvReading` item, and define a constraint `samplingRate` ≤ 10 Hz.
5. Define a port on `NodeController` that outputs `EnvReading` items.
6. Create `SensorNetwork` containing `SensorNode[1..*]` and a central collector part.
7. Connect each node's controller `pDataOut` to the collector using flow features.

Exercise 8.3: Battery charger module

Objective: Demonstrate modeling of flows, attributes, constraints, multiplicity, and cross-part interactions in a charging subsystem.

Estimated time: ~30 minutes

Problem description: A battery charging system consists of a `PowerConverter`, `ChargeController`, and multiple **Battery** units. Electrical power flows through the converter into the controller and then to individual batteries. Voltage and charge limits must be enforced to protect the batteries. This exercise shows how to model multi-part compositions with item flows, multiplicity, constraints, and internal actions.

Tasks:

1. Define `ElectricPower` as an item with attributes `voltage` and `current`.
2. Define `PowerConverter` with attribute `efficiency` and a port outputting `ElectricPower`.
3. Define `ChargeController` with `targetVoltage`, an action `regulateCharge`, and ports for `ElectricPower` flow.
4. Define `Battery` with attributes `voltage`, `capacity`, and `stateOfCharge` and a port receiving `ElectricPower`.
5. Add a constraint ensuring `battery.voltage ≤ targetVoltage`:
6. Create `BatteryCharger` with a `PowerConverter`, `ChargeController`, and `Battery[2..*]`
7. Model a flow splitter from the controller to the multiple Battery parts.

Exercise 8.4: 3D printer print-head assembly

Objective: Model a multi-stage material-flow subsystem emphasizing encapsulation, actions, and internal constraints.

Estimated time: ~25 minutes

Problem description: A print head in a 3D printer manages filament input, heating, and extrusion. Material flows from the feeder through a heater to a nozzle, where it is melted and extruded. Temperature must remain below safe operating limits. This exercise focuses on multipart flow pathways, internal operations, and encapsulation.

Tasks:

1. Define `Filament` as an item with attributes `materialType` and `temperature`.
2. Create `FilamentFeed`, `Heater`, and `Nozzle` part definitions, each with the required attribute.
3. Define the flow from `FilamentFeed` to `Heater` to `Nozzle`.
4. Add actions `meltMaterial` (in `Heater`) and `extrude` (in `Nozzle`), producing a `MeltFlow` item.
5. Add a constraint `temperature ≤ 250 °C` in `Heater`.
6. Create a `PrintHead` part exposing only `pFilament` and `pMeltFlow`, delegating internal flows.

Exercise 8.5: Weather station family

Objective: Model a reusable base weather-station component with flows, attributes, constraints, and actions.

Estimated time: ~20 minutes

Problem description: Weather stations continuously measure environmental data and communicate results for display and logging. A reusable `WeatherStation` definition includes a `controller`, `display unit`, and `power system`. It also produces `WeatherData` items that include `temperature`, `humidity`, and `pressure`. This exercise shows how to incorporate reusable data structures, flow features, actions, and constraints in a base component.

Tasks:

1. Define `WeatherData` with attributes temperature, humidity, and pressure.
2. Create part definitions `Controller`, `Display`, and `PowerUnit`.
3. Create `WeatherStation` with attributes `location` and `dataRate` (default 0.1 Hz).
4. Add an action `collectWeatherData` and a flow feature `dataOut: WeatherData`.
5. Add a constraint `dataRate ≥ 0.1 Hz`.
6. Instantiate or specialize additional stations if desired.

Exercise 8.6: Encapsulated pump controller using ports

Objective: Model an encapsulated pump-control subsystem with conjugated ports, internal delegation, actions, and constraints.

Estimated time: ~30 minutes

Problem description: A `PumpController` maintains fluid flow through an irrigation line. The controller interacts with internal parts (sensor, valve, and motor driver) using a shared flow-control interface. Each internal part uses the conjugated side of the same port definition. The controller exposes a single external command port that delegates internally to coordinate behavior. Constraints ensure the reported flow rate remains close to the commanded value.

Tasks:

1. Define `FlowCommand` and `FlowStatus` as items.
2. Define port `iFlowControl` with `in` item `flowCommand` and `out` item `flowStatus`.
3. Create `Sensor`, `Valve`, and `MotorDriver` part definitions, each with a conjugated `iFlowControl` port.
4. Create `PumpController` with an external port `pCommand: iFlowControl`.
5. Use delegation to connect `pCommand` to the `Valve` and `MotorDriver` internal ports.
6. Add actions `measureFlow` and `adjustValve`.
7. Add a constraint ensuring `flowStatus.rate ≈ targetRate ± 5%`.

Exercise 8.7: Robot arm with modular joint interfaces

Objective: Model a robotic arm with reusable joint modules, coordinated internal actions, and safe-motion constraints.

Estimated time: 30–60 minutes

Problem description: A robotic arm uses three joints (shoulder, elbow, and wrist), each with identical structure and behavior but independent motion limits. The arm receives a high-level `ArmCommand` item specifying target angles for each joint. The arm distributes the command to each joint and coordinates their motion while enforcing safe range limits. This exercise demonstrates modularity, item flows, reusable part definitions, internal action coordination, and encapsulation.

Tasks:

1. Define `ArmCommand` with `wristAngle`, `elbowAngle`, and `shoulderAngle` attributes.
2. Define `Joint` with attribute `angle`, action `moveJoint`, and a constraint `-90° ≤ angle ≤ +90°`.
3. Define `RobotArm` with parts `shoulder`, `elbow`, and `wrist` typed by `Joint`, and an external port `pArmCommands` carrying `ArmCommand`.
4. Add an action `moveArm` that invokes each joint's `moveJoint` using the corresponding angle.
5. Connect `pArmCommands` to each joint through delegation or flow features.
6. Demonstrate encapsulation by exposing only `pArmCommands` at the system boundary.

Chapter 9

The following exercises are designed to help you practice and internalize the structural modeling concepts introduced in this chapter on parts. Each activity focuses on a specific aspect of SysML v2 structural modeling: part definitions, usages, connections, ports, encapsulation, specialization, and cross-cutting relationships such as dependency and allocation. The exercises progress in complexity and duration, helping you gain hands-on experience building, refining, and connecting model elements that define robust, modular architectures.

Exercise 9.1: Modeling a drone assembly

Objective: Create a small structural model using *part definitions* and *part usages*, introducing feature multiplicity and attributes with different value counts.

Estimated time: 15–25 minutes

Background: You are modeling a simple quadcopter drone system composed of the following logical components:

- A `FlightController` that manages the drone's attitude and power distribution
- Four `Motor` assemblies, each driving a propeller
- A single `Battery` that provides electrical energy
- A set of `Accelerometers` used for stability and navigation

Your goal is to represent this architecture using *part definitions*, *usages*, and *multiplicities*.

Tasks

1. **Create part definitions** for the following components:
    - **FlightController** with attributes:
        - firmwareVersion : `String`
        - cpuLoad : `PercentageValue`
    - **Motor** with attributes:
        - rpm : `AngularVelocityValue`
        - thrust : `ForceValue`
    - **Battery** with attributes:
        - capacity : `ElectricChargeValue`
        - voltage : `ElectricPotentialValue`
    - **Accelerometer** with attribute:
        - accel : `Real`
    - **Quadcopter** as the top-level system part.
2. **Assemble the system** by defining the following *part usages* within **Quadcopter**:
    - controller : `FlightController`

- motor : `Motor [4]` — exactly four motors
- battery : `Battery [1]` — a single battery (implicit multiplicity)
- accelerometer : `Accelerometer [1..*]` — at least one sensor, possibly more

3. **Add simple attributes** to **Quadcopter** itself, such as:
   - mass : `MassValue`
   - maxPayload : `MassValue`
   - speed : `SpeedValue`
4. **Add connections** among appropriate parts within **Quadcopter** to represent physical or logical relationships (for example, power connections from the battery to each motor, or sensor data connections to the flight controller).

Exercise 9.2: Adding attributes, items, and flows

Objective: Extend part definitions to include different *feature types* (attributes, items, and flows) and model a simple thermal-fluid system using directed flow connections among parts.

Estimated time: ~20 minutes

Background: You are modeling a `CoolingSystem` that circulates coolant to remove heat from a subsystem and reject it to the environment.

The system consists of the following components:

- A `Pump` that circulates coolant through the circuit
- A `Radiator` that extracts heat from the coolant and transfers it to the external `Environment`
- A `Reservoir` that stores coolant and maintains system volume
- An `Environment` that absorbs the released heat energy

Your goal is to represent both the *physical structure* and the *flow of coolant and heat* through the system using item and flow features, as well as appropriate connectors.

Tasks

1. Define the items that flow through the system.
   - item `def Coolant` with attributes:
     - temperature : `TemperatureValue`
     - pressure : `PressureValue`
     - heatContent : `EnergyValue`
   - item `def Heat` with attribute:
     - energy : `EnergyValue`
2. Create part definitions for each subsystem:

Pump

- Attributes:
  - flowRate : `VolumeFlowRateValue`
  - powerDraw : `PowerValue`
- Flow features:
  - in item inFlow : `Coolant`
  - out item outFlow : `Coolant`

Radiator

- Attributes:
  - maxDissipation : `PowerValue` // rated cooling capacity
- Flow features:
  - in item hotCoolant : `Coolant`
  - out item cooledCoolant : `Coolant`
  - out item heatOut : `Heat`

Reservoir

- Attributes:
  - capacity : `VolumeValue`
  - level : `VolumeValue`
- Flow features:
  - in item returnCoolant : `Coolant`
  - out item supplyCoolant : `Coolant`

Environment

- Attributes:
  - ambientTemp : `TemperatureValue`
  - heatAbsorbed : `EnergyValue`
- Flow features:
  - in item absorbedHeat : Heat

3. Assemble the system in a *part def* CoolingSystem that owns the four parts:
   - pump : `Pump`
   - radiator : `Radiator`
   - reservoir : `Reservoir`
   - environment : `Environment`

4. Add connectors to represent the flow paths:
    - Connect the pump's `outFlow` to the radiator's `hotCoolant`.
    - Connect the radiator's `cooledCoolant` to the reservoir's `returnCoolant`.
    - Connect the reservoir's `supplyCoolant` back to the pump's `inFlow` — forming a closed coolant loop.
    - Connect the radiator's `heatOut` to the environment's `absorbedHeat` — completing the thermal path.
5. Type all attributes and flows using the appropriate quantity kinds and units from the *ISQ* and *SI* libraries.

Expected outcome: By completing this exercise, you will:

- Create item and part definitions with typed attributes and directed flow features
- Build a complete energy and material flow network, including an explicit environmental interface
- Model closed-loop coolant circulation and open-loop heat transfer
- Reinforce the physical meaning of system boundaries and environmental interactions, a key systems engineering concept

Exercise 9.3: Encapsulating the cooling system using ports

Objective: Convert the previously modeled `CoolingSystem` into an encapsulated subsystem that exchanges coolant and heat through ports typed by reusable port definitions, including realistic service/refill and waste-heat interfaces at the system boundary.

Estimated time: 30–45 minutes

Background: In *Exercise 9.2*, you built a closed coolant loop and modeled heat transfer to the environment. In this exercise you will:

- Replace direct flow connections with ports and port definitions (*interaction contracts*).
- Add service-entry and waste-heat ports at the system boundary.
- Use **delegation** to map system ports to the internal parts that actually implement them.
- Apply conjugation (~) correctly for part-to-part connections.

Tasks:

1. Define **interaction contracts** (**port definitions**): Create three port definitions to separate operational flows from service interfaces:
    - `iCoolantLoop` (operational loop inside the system)
        - `in item inCoolant : Coolant`
        - `out item outCoolant : Coolant` (used for the closed loop among `Pump`, `Radiator`, `Reservoir`)

- `iCoolantRefill` (service/refill at the boundary → into `Reservoir`)
  - `in item refillCoolant : Coolant` (used by the system boundary and `Reservoir` for adding coolant)
- `iHeatTransfer` (waste heat leaving the system to the environment)
  - `out item heatOut : Heat` (used for rejecting waste heat through the system boundary)

2. Update internal part definitions to use ports.
3. Keep attributes from Exercise 9-2. Add/adjust ports as follows:
   - Pump
     - pLoop : `iCoolantLoop` (provides outCoolant to the loop)
   - Radiator
     - pLoop : `~iCoolantLoop` (receives coolant from Pump; returns cooled outCoolant)
     - pHeat: `iHeatTransfer`
   - Reservoir
     - pLoop : `iCoolantLoop` (returns/supplies loop coolant)
     - pRefill : `iCoolantRefill` (receives `refillCoolant` from outside the system)
   - Environment
     - Remove this part from the system because now the waste heat will flow to the system port pHeat
4. Define the encapsulating subsystem ports on `CoolingSystem`:
   - `pCoolant : iCoolantRefill` (external service port to add coolant).
   - `pHeat : iHeatTransfer` (external waste-heat port to reject heat).

   These are system boundary ports, distinct from the internal part ports.
5. Declare internal parts:
   - `pump : Pump`
   - `radiator : Radiator`

    - `reservoir : Reservoir`
    - `(removed) environment : Environment`
6. Add delegations and internal connections (corrected):
    - Delegations (owner ↔ internal; same orientation, no conjugation)
        - Delegate `CoolingSystem.pCoolant` ⟶ `reservoir.pRefill` (service refill goes to the Reservoir).
        - Delegate `CoolingSystem.pHeat` ⟶ `radiator.pHeat` (waste heat is exposed at the system boundary).
    - Internal connections (between parts; connect an unconjugated port to a conjugated port)
        - Connect `pump.pOut` ⟶ `radiator.pIn`.
        - Connect `radiator.pOut` ⟶ `reservoir.pIn`.
        - Connect `reservoir.pOut` ⟶ `pump.pIn`.

Exercise 9.4: Modeling a warfare system-of-systems (WSoS) architecture

Objective: Develop a *taxonomy* of platform types using specialization and assemble them into a composite WSoS architecture that captures multiplicities, optional components, and intersystem relationships.

Estimated time: 45–60 minutes

Background: You are asked to model a **WSoS** enterprise, a large-scale architecture integrating multiple heterogeneous platforms across operational domains.

There are several *kinds* of platforms, including:

- **Ground-based platforms**
- **Air-based platforms**
- **Sea-based platforms**
- **Space-based platforms**

Each platform type shares certain structural or behavioral characteristics but specializes in its own operating environment. Your goal is to define this taxonomy and assemble a representative WSoS instance.

The `WSoS` includes:

- 8 Abrams tanks (a specialized kind of ground-based platform)
- 1 airborne warning and control system (**AWACS**) aircraft (a specialized kind of air-based platform) that collects and processes sensor data from satellites and transmits information to the Mission Control Station
- 10–20 surveillance satellites (a specialized kind of space-based platform) that provide raw intelligence to the AWACS

- An optional aircraft carrier (a specialized kind of sea-based platform) that can host up to 6 F-35 fighters
- A Mission Control Station (a specialized kind of ground-based platform) that commands the tanks, carrier, and fighters

Tasks:

1. Define the platform taxonomy:
    - Create an abstract `Platform` part definition.
    - Specialize it into:
        - `GroundPlatform`
        - `AirPlatform`
        - `SeaPlatform`
        - `SpacePlatform`
    - Give each kind of platform descriptive attributes such as `range`, `crewSize`, or `payloadCapacity`.
2. Elaborate the taxonomy by adding the specific platform types as specializations of their base types:
    - `AbramsTank`
    - `MissionControlStation`
    - `AWACS`
    - `F35`
    - `AircraftCarrier`
    - `SurveillanceSatellite`
3. Assemble the system-of-systems: Create a part definition `WarfareSoS` that includes:
    - `tank : AbramsTank [8]`
    - `awacs : AWACS [1]`
    - `satellite : SurveillanceSatellite [10..20]`
    - `carrier : AircraftCarrier [0..1] // optional`
    - `airwings : F35 [0..6] // fighters hosted on aircraft carrier`
    - `control : MissionControlStation [1]`
4. Add connectors to represent key relationships:
    - `satellite` → `awacs` (intel connection)
    - `awacs` → *control* (processed intel connection)
    - *control* → `tank`, *control* → `carrier`, *control* → `fighter` (command connections)
5. Model information flows or commands. Show these flows on a different general view. Define representative *connections* between parts:
    - Satellites → `AWACS`: Intel flow.

- `AWACS` ⟶ `MissionControlStation`: Processed intel.
- `MissionControlStation` ⟶ tanks, carrier, F-35s: Command flows.

6. Create a new general view to show the "connected view" as an interconnection view.
   Diagramming requirement: Create and show *three separate views (diagrams)* to represent distinct modeling perspectives:
7. *Platform taxonomy diagram*: The specialization hierarchy of `Platform` and its subtypes.
8. *Composition diagram*: The internal structure of WSoS, showing the parts and their multiplicities.
9. *Connected architecture diagram*: The connections and information flows among the parts in WSoS.

Each diagram should clearly reflect its own level of abstraction (type, structure, and interaction) and have a comment describing what it is trying to show.

Exercise 9.5: Specializing the WSoS for Mission X (redefinition and subsetting)

Objective: Create a mission-specific specialization of the `WSoS` that tightens multiplicities, swaps in explicit variants, and partitions collections using subsetting. Practice redefinition for attributes/ports and specialization for type variants.

Estimated time: 45–75 minutes

Background: You previously modeled a generic `WSoS` with variable and optional multiplicities (*Exercise 9.4*). In real programs, an operational mission often narrows those options into a concrete configuration. In this exercise, you will define `Mission X` as a specialization of WSoS that fixes quantities, introduces platform variants, and partitions satellite kinds.

`Mission X` specifics:

1. Exactly 15 surveillance satellites total:
   - 10 are IR-sensor satellites.
   - 5 are optical satellites.
2. The aircraft carrier is present and hosts exactly 4 `F-35` aircraft, and the fighters are an **F-35b6** variant (six internal missiles instead of the standard four).
3. All Abrams tanks are the `M1A2` variant.

Tasks:

1. Update the taxonomy (specialization).
   - Specialize `SurveillanceSatellite` into:
     - `IRSurveillanceSatellite`
     - `OpticalSurveillanceSatellite`
   - Specialize *F35* into `F35b6` (document how it differs; for example, `missilesInBay = 6`).
   - Specialize `AbramsTank` into `AbramsM1A2` (capture distinguishing attributes or capabilities).

2. Redefine attributes and/or ports where needed:
   - If your baseline *F35* has an attribute like `missile : Missile[*]`, redefine it on `F35b6` to `[6]`.
   - If your tanks/fighters share common command or status ports, consider redefining those ports on the variants only if the interface contract changes (otherwise inherit as is).
3. Create the `Mission X` system by specialization:
   - Define `MissionX` as a specialization of `WarfareSoS` (a more constrained system type).
   - Redefine multiplicities in `MissionX` to match the mission's fixed counts:
     - `satellite` exactly 15
     - `Carrier` exactly 1
     - `fighter` exactly 4
     - `tank` (Abrams) keep at 8, but all must be the `M1A2` variant
   - Where you kept ranges in WSoS, tighten them in `MissionX` to exact values.
4. Subset collections to partition satellites by kind:
   - Introduce two subset features of the `satellite` collection in `MissionX`:

     ```
     part satellite[15] : Satellite;part irSat[10] : IRSurveillanceSatellite
     subsets satellite;part optSat[5] : OpticalSurveillanceSatellite subsets
     satellite;
     ```

   - Ensure the two subsets are disjoint and their cardinalities sum to the redefined total (15).
5. Redefine types on parts where variants are required:
   - Redefine `fighter` to be of type `F35b6` in `MissionX` (multiplicity 4).
   - Redefine `tank` to be of type `AbramsM1A2` in `MissionX` (multiplicity 8).
   - Ensure the `carrier` is present (multiplicity 1) and, if relevant, subset its internal `airWing` (or equivalent) to host exactly four `F35b6` fighters.
6. Preserve or tighten inter-system connections:
   - Retain the core relationships from WSoS (such as satellites → AWACS intel, AWACS → MissionControl processed intel, and `MissionControl` → tanks/carrier/fighters commands).
   - If you named connections or ports in *Exercise 9.4*, keep them but constrain multiplicities/participants in `MissionX` (for example, commands to exactly eight `M1A2` tanks and four `F35b6` fighters).

Deliverables:

- *Type (taxonomy) view*: Show the variant classes (`IRSurveillanceSatellite`, `OpticalSurveillanceSatellite`, `F35b6`, `AbramsM1A2`) as *specializations* of the baseline types.

- `MissionX` *composition view*: Diagram the `MissionX` structure with redefined multiplicities and subset partitions (`irSat` and `optSat`), plus the specialized types for tanks and fighters.
- `MissionX` *connected view*: Reuse the WSoS connections, now scoped to the `MissionX` participants (for example, intel from 15 satellites to the single AWACS; commands from Mission Control to eight M1A2, one carrier, and four F35b6 fighters).

Exercise 9.6: Requirements, satisfaction, and connections

Objective: Create an integrated model of an autonomous warehouse **AGV** (**automated guided vehicle**) that links requirements to structural and behavioral features. Define requirements, model the AGV's internal composition and connections, and show how specific parts satisfy system-level requirements.

Estimated time: 1–2 hours

Background: This exercise extends structural modeling to include *requirements traceability*. The AGV must operate safely and efficiently in a warehouse environment, coordinating propulsion, sensing, computation, power management, and docking subsystems.

You will:

- Define and document requirements directly in the model.
- Create part definitions and their relationships.
- Model ports, flows, and connectors among subsystems.
- Link requirements to realizing features through `satisfy` relationships.

This exercise integrates concepts from *Chapter 4* (requirements and documentation), *Chapter 7* (information and flows), and *Chapter 8* (parts, ports, and composition).

System context: Model the AGV as a system-level part definition that includes internal parts such as:

- `PowerTrain`: Converts electrical energy to torque for propulsion.
- `BatteryPack`: Stores and supplies electrical energy.
- `ThermalMgmt`: Regulates component temperature.
- `ComputeModule`: Performs navigation and motion control.
- `SensorSuite`: Detects obstacles and provides situational data.
- `NetworkModule`: Manages communications.
- `BrakingSystem`: Provides normal and emergency stopping capability.
- `DockingStation`: Recharges the AGV.

Additional subsystems may be added as needed.

Requirements: Create the following as requirement usages inside the AGV system. (Refer to *Chapter 4* for requirement notation if needed.)

Requirement name	Requirement usage
`rThroughput`	The AGV shall sustain ≥ 40 pallets/hour over a 1-hour interval.
`rSafetyStop`	The AGV shall achieve a full stop within 0.8 s from 1.5 m/s upon obstacle detection.
`rObstacleDetection`	The AGV shall detect a 0.2 m obstacle at a range of at least 10 m with a detection latency of less than 100 ms.
`rEndurance`	The AGV shall operate ≥ 8 hours per charge.
`rThermalProtection`	No critical component temperature shall exceed 85 °C.
`rPowerBudget`	Average electrical power shall not exceed 600 W.
`rCommsLatency`	Network latency shall remain ≤ 50 ms (p95).
`rDockingAutonomy`	The AGV shall autonomously dock and begin charging within 60 s of a low-battery alert.
`rPayloadMass`	The AGV shall safely carry ≤ 500 kg payload.

Table 9.3 – AGV system requirements and requirement usages

Add additional requirements if needed for completeness.

Tasks:

1. *Define the part definitions and their features*: Create part definitions for each subsystem, including attributes, actions, and quantitative constraints that contribute to requirement satisfaction (such as torque, capacity, latency, brake force, and temperature limit).
2. *Add ports and flows*: Define the items that flow between parts (for example, obstacle alerts, brake commands, telemetry, cooling air, or charging current) and create port definitions to represent them. Add connectors between parts to show the exchange of energy, control, or information in *Step 4*.
3. *Establish satisfaction relationships*: Use `satisfy` relationships between concrete part usages and requirement usages. Each requirement should be satisfied by one or more features such as an attribute, action, or constraint that realizes the intended behavior or performance.
4. *Connect the system structurally and behaviorally*: Complete the internal connections showing how data, commands, and power flow through the AGV. Include port directionality and multiplicity where relevant. Connections should demonstrate how collaboration among parts enables the AGV

to meet its requirements. (Optional: add dependencies if you wish to show purely conceptual design reliance.).

5. *Produce four complementary views*:
    - *Requirements view*: Lists and organizes all requirement usages with brief text. This can be a requirements table (preferred) or a General View.
    - *Composition view*: Shows the structural hierarchy of parts and their multiplicities.
    - *Connected view*: Illustrates connectors and flows of items, commands, and energy among the parts.
    - *Satisfaction view*: Shows `satisfy` relationships linking requirements to the structural or behavioral features that realize them.

Exercise 9.7: Allocating mission-thread functions

Objective: Model a *mission thread* called Defend via remote tracking that captures a cooperative defensive engagement between a radar site and an offshore ship. You will define the mission's logical functions, model the major physical systems, and then allocate each mission-level function to the subsystem responsible for performing it.

Estimated time: 90–120 minutes

Background: In this exercise, you will model a coordinated defensive engagement based on the **F2T2EA kill chain** (find → fix → track → target → engage → assess). A remote radar site detects and tracks an enemy missile and transmits tracking data to an offshore ship equipped with a **C4 system (command, control, communications, and computing)** and a weapon system capable of launching anti-missile interceptors. Your task is to represent both the *logical mission thread* and the *physical architecture*, then allocate the abstract functions to the concrete subsystems that execute them.

> **Note**
>
> **What is a mission thread?**
>
> A **mission thread** is a structured scenario that traces the sequence of actions, interactions, and information flows required to accomplish a specific operational objective. Unlike a simple process or use case, a mission thread spans multiple systems and organizational boundaries, showing how collaborating elements (human, software, hardware, and external systems) work together to achieve mission success. In systems modeling, mission threads provide the logical backbone that connects *what must happen* (functions, decisions, and exchanges) to *who performs it* (systems, subsystems, and components). They are particularly valuable in system-of-systems engineering, where coordinated behavior among distributed participants determines overall mission effectiveness. For further background, see the U.S. Department of Defense's *Mission Engineering Guide* (available at `https://ac.cto.mil/wp-content/uploads/2023/11/MEG_2_Oct2023.pdf`).

Mission-thread functions: The logical model should include at least the following mission-level actions. Each corresponds to a step in the defensive sequence:

Action	Purpose/Description
Track target	Detect and maintain track on the incoming threat.
Merge tracks	Combine radar and ship sensor data into a single coherent track.
Identify threat	Classify and confirm the target as hostile.
Determine threat level	Assess threat intent and priority based on trajectory and behavior.
Select countermeasure	Choose the appropriate defensive weapon or tactic.
Target enemy	Compute fire-control solutions for the selected countermeasure.
Launch missile	Command the interceptor missile launch.
Guide missile	Control and update the interceptor's flight path.
Assess engagement	Evaluate results and update situational awareness.

Table 9.4 – Mission-thread actions and their purposes

Tasks:

1. Define the logical architecture:
    - Create an occurrence definition for the *logical mission thread* (such as `MissionThread_Logical`).
    - Include the preceding actions as *actions usages* within that occurrence definition.
    - Define any supporting *item definitions* or *flows* (such as `TargetTrack`, `FireControlData`).
2. Define the physical architecture:
    - Create the part definitions for the physical systems and subsystems that participate in the mission.
    - Create the `MissionSystem_Physical` occurrence definition as the context containing part usages for the principal systems and other participants that work together in the mission:
        - `RadarSite`: Performs detection and tracking.
        - `Ship`: Host platform containing the `C4System` and `WeaponSystem` as parts.
        - `C4System`: Composed of `CommandAndDecision`, `Tracking`, `Targeting`, and `WeaponsManagement` subparts
        - `WeaponSystem`: Includes `Launcher` and `Interceptor`
    - Add in the `Defended Site` and `Enemy Missile` as participants in the overall mission thread.
3. Model system connections and flows:
    - Add connections or ports showing data exchange between the radar, C4 system, and weapon system (such as tracking data, fire-control commands).

4. Allocate mission functions:
    - Allocate each logical mission action to one or more mission system parts, such as allocating the `Track Target` action to `RadarSite` and `C4System.tracking`.
    - Ensure allocations connect parts or features of parts, not definitions.
5. Create views: Prepare the following complementary views:
    - *Mission-thread view*: Shows logical actions and sequencing.
    - *System context view*: Shows the overall system-of-systems elements (radar site, ship, coastal defended area, enemy missile) and how they connect.
    - *Physical architecture view*: Shows systems and subsystems, their properties, and how they connect.
    - *Allocation view*: Shows mappings between logical functions and physical parts.

Exercise 9.8: Greenhouse environmental control system

Objective: Develop an integrated SysML v2 model of a **greenhouse environmental control system** (**GECS**) that demonstrates complete structural, behavioral, and allocation modeling, including parts, ports, flows, multiplicities, interfaces, and cross-domain allocations.

Estimated time: 2–3 hours

Background: You are modeling an automated greenhouse designed to maintain optimal growing conditions for multiple plant bays. The system regulates temperature, humidity, and soil moisture using a combination of sensors, actuators, and control software.

This exercise draws on all major modeling concepts introduced in this chapter:

- **Part definitions** and usages
- **Attributes**, items, and flows
- **Ports** and **encapsulation**
- Connections and interfaces
- **Multiplicity** of parts and features
- Specialization and Subsetting
- Satisfaction and allocation relationships

System description: GECS monitors and regulates the environment across multiple plant bays. The major subsystems are mentioned in the following table:

Subsystem	Purpose/Description
`EnvironmentController`	The supervisory controller that coordinates all sensors and actuators; executes control algorithms.
`ClimateSensors`	Measures air temperature, humidity, and CO_2 concentration.
`SoilSensors`	Measures soil moisture in each plant bay.

Subsystem	Purpose/Description
`Heater`	Raises ambient temperature within the greenhouse.
`Humidifier`	Increases relative humidity.
`IrrigationSystem`	Waters each plant bay as required; includes valves and water reservoir.
`VentilationSystem`	Exhausts hot or humid air to maintain equilibrium.
`PowerSupply`	Provides electrical energy to all subsystems.

Table 9.5 – GECS subsystems and their purposes

System-level requirements: Define the following requirement usages (referring to *Chapter 4* for syntax guidance):

ID	Requirement statement
`R-1`	The system shall maintain internal air temperature within ±2 °C of the setpoint.
`R-2`	The system shall keep relative humidity between 40 % and 70 %.
`R-3`	The system shall irrigate soil when soil moisture falls below 30%.
`R-4`	The system shall log environmental data every 5 minutes.
`R-5`	The system shall operate for 8 hours on backup power.

Table 9.6 – GECS system-level requirements

Each of these requirements will later be *satisfied* by the appropriate subsystem(s) or part features.

Task 1: Define logical architecture:

1. Create a part definition `Greenhouse_Logical`, representing the abstract control functions:
    - `action senseEnvironment`
    - `action analyzeConditions`
    - `action controlClimate`
    - `action irrigatePlants`
    - `action logData`
2. Define any supporting item definitions, for example:
    - item `def EnvData { temperature : TemperatureValue; humidity : Real; }`
    - item `def MoistureReading { moisture : Real; }`

These actions will later be *allocated* to the physical components that realize them.

Task 2: Define physical architecture:

1. Create part definitions for each subsystem listed in the preceding table.
2. Add appropriate attributes (for example, `powerConsumption`, `capacity`) and ports typed by interfaces such as `iClimateData` and `iControlSignal`.
3. Use feature multiplicities where appropriate, for example, multiple `SoilSensor[1..*]` parts.
4. Assemble these into a part definition `Greenhouse_Physical` containing all major parts, connected through ports for data and power.

Task 3: Encapsulation and ports:

1. Ensure each subsystem exposes only well-defined ports for interaction, hiding its internal structure.
2. Use delegation to connect external system ports (such as `pClimateData`, `pCommand`) to internal subsystem ports.
3. Create port definitions for `iSensorData`, `iControlSignal`, and `iPower`.

Task 4: Define satisfaction relationships:

1. Relate requirements `R-1`–`R-5` to the subsystem(s) that satisfy them, for example:
   - R-1 is satisfied by `EnvironmentController`, `Heater`, and `VentilationSystem`
   - R-5 is satisfied by `PowerSupply`.
2. Use satisfy requirement usages rather than allocation (parts → requirements).

Task 5: Allocation of logical to physical:

1. Allocate each logical action from `Greenhouse_Logical` to the physical subsystem(s) performing it:
   - `senseEnvironment` → `ClimateSensors`, `SoilSensors`
   - `analyzeConditions` → `EnvironmentController`
   - `controlClimate` → `Heater`, `Humidifier`, `VentilationSystem`
   - `irrigatePlants` → `IrrigationSystem`
   - `logData` → `EnvironmentController`
2. Verify allocations are between *features of part usages*, not type-level elements.

Task 6: View:

Prepare the following complementary diagrams:

Diagram	**Purpose**
Requirements view	Shows the defined requirements and their `satisfy` relations.
Composition view	Depicts `Greenhouse_Physical` with all major parts and multiplicities.

Diagram	Purpose
Flow/Connection view	Shows ports, connections, and flow directions (e.g., sensor data, control signals, power).
Allocation view	Maps logical functions to physical parts (logical → physical).

Table 9.7 – Complementary diagrams and their purposes

Chapter 10

The following exercises move from defining reusable types and concrete individuals to reasoning about occurrences, temporal extent, and clocks. You will work hands-on with structure, behavior, and time, creating parts and items, binding actions to participants, sequencing events, applying constraints, and using clocks to give your models a sense of "when." The goal is not just to practice syntax, but to build intuition for how SysML v2 represents existence, behavior, and time.

Exercise 10.1: Definitions, usages, and individuals in context

Objective: Practice distinguishing between *definitions*, *usages*, and *individuals* by modeling a small structural and informational system and then instantiating it with individuals.

Estimated time: 40–50 minutes

Problem description: A simple sensor system consists of a sensing element, a processor, and discrete data packets that flow between them. The goal is to model the structural types (parts), the informational type (item), and their composition into a system definition. You will then create individuals that represent specific real-world components and data, assemble a concrete system individual from those individuals, and reflect on how model-level constructs classify real-world instances.

Tasks:

1. Define the part and item types:
    - Create `Sensor` as a part def with attribute `sensitivity : Real`.
    - Create `Processor` as a part def with attribute `clockRate : FrequencyValue`.
    - Create `DataPacket` as an item def with attributes `payload : String` and `timestamp : TimeInstantValue`.
2. Assemble the system definition:
    - Create `SensorSystem` as a part def containing parts:
        - `sensor : Sensor`
        - `cpu : Processor`
    - Add a connector or flow of `DataPacket` from `sensor` to `cpu`.
3. Define individuals for the components and data:
    - Create `Sensor_A` as an individual part def specializing `Sensor` (`part def :> Sensor`), with `sensitivity= 0.8`.
    - Create `CPU_1` as an individual part def specializing `Processor` (`part def :> Processor`), with `clockRate= 3.5 [GHz]`.
    - Create `Packet_1` as an individual item def specializing `DataPacket` (`item def :> DataPacket`), with `payload = "TempReading"`.

4. Assemble the real system individual:
    - Create `SensorSystem_2025` as an individual part def specializing `SensorSystem` (`part def :> SensorSystem`) that:
        - redefines `sensor : Sensor_A`
        - redefines `cpu : CPU_1`
        - Includes `Packet_1` as the current data flow
5. Reflect on the model:
    - Explain how the definitions, usages, and individuals relate.
    - Identify which elements exist only in the model and which are intended to represent concrete entities in the real world.

Exercise 10.2: Occurrences in a mission thread: Coordinating drone deliveries

Objective: Model a small delivery system in which structural participants (dispatcher and drones) coordinate informational occurrences (orders) through mission-level occurrences, reinforcing the distinction between structural and informational occurrences over time.

Estimated time: 60–75 minutes

Problem description: A logistics company operates a fleet of autonomous drones under the control of a central dispatcher. Each drone handles one delivery order at a time. The dispatcher assigns orders, monitors progress, and records completion. Orders form the informational layer, while the dispatcher, drones, and delivery system form the structural layer. The goal is to connect these layers through mission-level occurrences that describe how deliveries unfold.

Tasks:

1. Define the structural participants:
    - Create a `Dispatcher` definition responsible for issuing delivery orders and monitoring status.
    - Create a `Drone` definition that can receive an order, navigate to a destination, and deliver a package.
    - Assemble them into a `DeliverySystem` definition containing one dispatcher and multiple drones.
    - Decide how the dispatcher and drones communicate (for example, via ports, connections, or item flows).
2. Define the informational occurrence:
    - Model `Order` as an informational item that the dispatcher sends to a drone.
    - Give `Order` attributes such as:
        - `orderId` or tracking number

        - Delivery destination
        - Package weight
        - Priority level
        - Delivery window or deadline (for example, `DurationValue`)
    - Consider how this information could support later behavioral and temporal analysis.
3. Model the mission occurrence:
    - Create a reusable `DeliveryMission` occurrence definition that coordinates one dispatcher, one drone, and one order.
    - Capture the logical action sequence: the dispatcher assigns the order, the drone navigates, and then the drone delivers.
    - Represent how the order flows from dispatcher to drone as part of this mission thread.
4. Compose the fleet operation:
    - Define a higher-level occurrence such as `FleetOperation` that includes multiple `DeliveryMission` occurrences running in parallel.
    - Bind each mission to a different drone within the system, ensuring each drone handles only one order at a time.
    - Decide whether any synchronization or sequencing is required between missions (for example, all drones must complete before new orders are issued).
5. Instantiate real individuals:
    - Create individuals for at least one dispatcher, two drones, and two unique orders.
    - Assemble specific mission occurrences (such as `MissionA`, `MissionB`), each pairing one order with one drone.
    - Reflect on how these individuals represent real-world entities and how their existence relates to definitions and usages.
6. Reflect on the model by answering the following questions:
    - Which elements represent structural occurrences and which represent informational occurrences?
    - How does the `DeliveryMission` occurrence capture runtime behavior?
    - If the company added a second dispatcher or allowed drones to queue multiple orders, what parts of the model would you refine?

Exercise 10.3: Modeling temporal extent: A drone's delivery cycle

Objective: Use time slices, snapshots, and temporal constraints to model how a drone's delivery mission unfolds over time and to reason about durations, sequencing, and performance.

Estimated time: 60–75 minutes

Problem description: Each drone executes a delivery mission in three phases: pre-flight checks, in-transit flight, and post-delivery wrap-up. You want to represent this as a single occurrence that spans the entire operation, decomposed into time slices and punctuated by key snapshots. The model should support analysis of durations and timing constraints that govern mission performance.

Tasks:

1. Define the occurrence:
    - Create an occurrence def `DeliveryCycle` representing the entire mission of one drone.
    - Include three time slices corresponding to *pre-flight*, *in-transit*, and *post-delivery* phases.
    - Sequence them explicitly using `first … then …` so their ordering is clear.
2. Add snapshots:
    - Define snapshots such as:
        - Lift-off instant (start of in-transit)
        - A mid-flight checkpoint or location update
        - The instant the package is delivered (end of in-transit)
    - Assign explicit time values to at least one snapshot (for example, flight begins at 120 s).
3. Apply temporal constraints:
    - Use `TimeOf()` and `DurationOf()` to express measurable properties, such as:
        - The in-transit slice must last at least 3 minutes but no more than 20 minutes.
        - The total duration from the start of pre-flight to the end of post-delivery equals the sum of slice durations.
    - Ensure the sequence is well-formed (each slice's end precedes the next slice's start).
4. Incorporate special snapshots:
    - Reference the `startShot` and `endShot` of the overall `DeliveryCycle` and of the in-transit slice.
    - Constrain these so they align correctly with your explicitly named snapshots.
5. Analyze and extend:
    - Identify performance metrics you can check (total mission time, turnaround time, and so on).
    - Reflect on how snapshots could be used to record telemetry or compliance data in simulation or digital-thread contexts.
6. Reflect on the model by answering the following questions:
    - What does each time slice represent in the lifecycle of the delivery?
    - How do snapshots differ from slices in what they capture?
    - How can `DurationOf()` and `TimeOf()` help validate timing requirements?
    - How would adding a second drone performing deliveries concurrently change your temporal reasoning?

Exercise 10.4: Modeling local clocks and synchronization

Objective: Explore local and global time references in a distributed system by modeling components with independent clocks, then expressing relationships and synchronization constraints among them.

Estimated time: 75–90 minutes

Problem description: In a distributed delivery system, each drone operates on its own local clock that may drift relative to the dispatcher's universal system clock. The dispatcher schedules orders against global time, while drones use local clocks for internal timing (pre-flight delays, in-transit durations, and so on). The goal is to model these clocks, represent behaviors that use them, and express temporal constraints tying local events back to a global reference.

Tasks:

1. Define clocks and their contexts:
    - Create a base part definition that introduces a local clock (an instance of `Time::Clock`) so each instance measures time independently.
    - Create specialized parts (for example, `Dispatcher`, `Drone`) that either inherit or redefine this local clock.
    - Use the universal clock for `Dispatcher` and local clocks for each `Drone`.
2. Model behavior with timing:
    - Give each `Drone` a behavior that includes a local delay (for example, accept after 30 [s] before takeoff).
    - For `Dispatcher`, define a behavior that waits until a scheduled absolute time (for example, "2026-05-17T06:00:00Z") before sending orders, using the universal clock.
3. Compose the system:
    - Create a `RealTimeDeliverySystem` with one dispatcher and multiple drones.
    - Model coordination such that the dispatcher's global "send order" event precedes each drone's local "begin delivery" event.
4. Express temporal relationships:
    - Add constraints ensuring each drone starts delivery within an allowable window after the dispatcher's send event, measured on the universal clock (for example, within 10 seconds).
    - Constrain the local `DurationOf(DeliveryCycle)` for each drone to be consistent across runs.
5. Experiment with synchronization:
    - Extend the model with a synchronization point: a global "all deliveries complete" event.
    - Consider how local clock drift affects this, and decide whether to model it as a global occurrence or via constraints aligning local events to a reference.
6. Reflect on the model by answering the following questions:
    - Why is it useful to model local clocks instead of assuming a single universal time?

- How do absolute time triggers (`accept at …`) differ from relative triggers (`accept after …`)?
- How might time drift or loss of synchronization be represented and analyzed?
- How would you represent periodic resynchronization (for example, a drone syncing with GPS)?

Exercise 10.5: Mini hydraulic press: Force, area, and pressure

Objective: Model a small hydraulic press using occurrences and calculations to derive pressure and output force from applied force and piston areas, then verify constraints on allowable pressure and required output force.

Estimated time: 20–30 minutes

Problem description: A bench-top hydraulic press has a `MasterPiston` (where a user applies force) and a `SlavePiston` (which delivers amplified pressing force) connected through an incompressible fluid in a `HydraulicChamber`. You want to compute chamber pressure from an applied force and piston area, compute the resulting output force, and check safety and performance constraints. The model should include a clear problem statement, calculations, and constraints that reflect the physics.

This system is represented schematically in *Figure 10.8*.

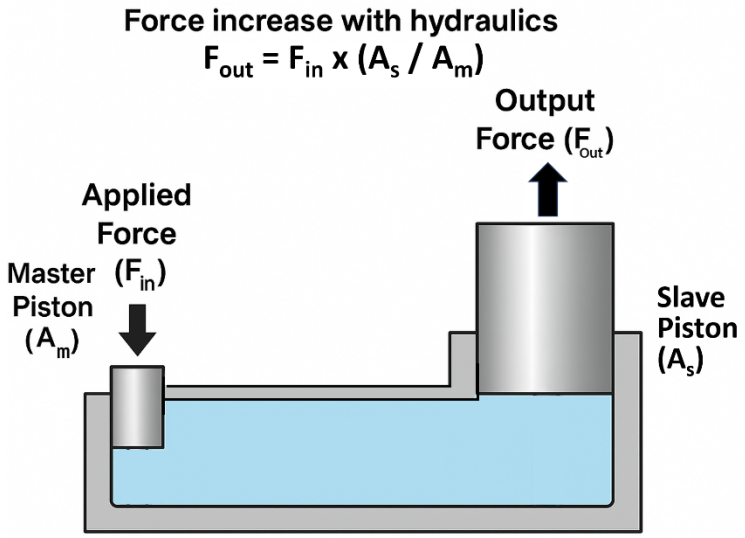


Figure 10.8 – Hydraulic press

Tasks:

1. Describe the system:
   - Write a brief natural-language problem statement for `HydraulicPress` explaining what it is and what is being computed, and store this in its `doc` property.

2. Model structure (parts):
    - Create `HydraulicPress` as the composite system of interest.
    - Create `MasterPiston` with face area A_m.
    - Create `SlavePiston` with face area A_s.
    - Create `HydraulicChamber` sharing the same pressure on both pistons.
3. Model information/items:
    - Define the following items: `AppliedForce`, `ChamberPressure`, and `OutputForce`.
4. Define calculations (actions):
    - Create `ComputePressure` to calculate pressure ($P = F_{in} / A_m$).
    - Create `ComputeOutputForce` to calculate output force ($F_{out} = P * A_s$).
    - Include any necessary unit conversions (for example, cm^2 to m^2).
5. Set given values:
    - Choose simple values, for example:
        - $F_{in} = 300$ N
        - $A_m = 4.0$ cm^2
        - $A_s = 16.0$ cm^2
        - $P_{max} = 8$ MPa
        - $F_{target} = 900$ N
    - Document these chosen values in the model so they can be sanity-checked.
6. Add constraints:
    - Add a constraint enforcing $P \le P_{max}$ (pressure limit).
    - Add a constraint enforcing $Fout \ge F_{target}$ (meets goal).
7. Explain the dependency chain:
    - Add a short paragraph (for example, as a note or doc text) explaining how `AppliedForce` + master area produce pressure, how pressure produces `OutputForce`, and how both constraints depend on those derived values.

Exercise 10.6: Temporal extent: Thermal cycle of a metal furnace

Objective: Apply lifetime, time slices, snapshots, and temporal constraints to model a complete thermal cycle for a metal furnace, from loading to unloading.

Estimated time: 45–60 minutes

Problem description: A metal heat-treatment furnace runs a repeated cycle: heating, soaking, and cooling a metal part. One complete `ThermalCycle` begins when the part is loaded and ends when it is fully cooled

and removed. During this lifetime, the furnace passes through distinct phases, and specific events occur at key instants. The aim is to capture this as an occurrence with well-defined slices, snapshots, durations, and constraints that support verification and analysis.

Tasks:

1. Define the occurrence:
    - Create an occurrence def `ThermalCycle` representing one complete heating run from part loaded to part unloaded.
2. Partition the lifetime into slices:
    - Define slices for *heating*, *soak*, and *cooling*.
    - Sequence them explicitly (for example, first heating, then soaking, then cooling).
3. Create snapshots:
    - Define snapshots `tStart`, `tTargetReached`, `tHoldEnd`, and `tUnloaded`.
    - Embed these snapshots in the temporal structure of the slices.
    - Use `assert` constraints with `TimeOf(...)` to relate each snapshot to the start and end shots of the appropriate slices (for example, `tTargetReached` lies between `heating.startShot` and `heating.endShot`).
4. Reason about durations:
    - Constrain slice durations, for example:
        - `DurationOf(heating) = 20 [min]`
        - `DurationOf(soak) = 60 [min]`
        - `DurationOf(cooling) = 30 [min]`
    - Constrain the total duration to equal the time between `tStart` and `tUnloaded`.
5. Add a verification constraint:
    - Write a constraint that checks that the soak period is longer than both heating and cooling individually.
6. Reflect on the model:
    - Consider what performance or compliance metrics can be checked using this model (for example, total cycle time, minimum soak time).

Chapter 11

This section provides exercises for you to practice the concepts and techniques discussed in this chapter. The time estimates are for general guidance only.

Exercise 11.1: Identifying actions in a lab centrifuge cycle

Objective:

Recognize where actions occur in a simple physical process and classify them using SysML v2 action concepts.

Estimated time: 15–20 minutes

Tasks:

- Consider a **bench-top lab centrifuge** used to spin blood samples. The centrifuge:
  - Accepts tubes placed in a rotor
  - Closes and locks the lid
  - Accelerates to a target speed
  - Spins for a specified duration
  - Decelerates and unlocks the lid
- *Identify at least six distinct behaviors in this process (for example, lock lid, accelerate rotor).*
- For each behavior, classify it as one of:
  - Basic *action*
  - *Assignment* action (changing a value)
  - *Send* or *accept* action (if you introduce a simple signal)
  - *Structured control action (for example, if or loop, if applicable)*
- *For each behavior, identify which part would perform it (for example,* `RotorController`*, LidLock, UserInterface).*
- Identify at least one spot where behaviors should be *sequenced* and one spot where they could reasonably occur *concurrently*.

Expected work products:

- A SysML package containing:
  - A list of at least three part definitions (for example, `CentrifugeController`, Rotor, `LidMechanism`)
  - A table or structured list of named actions with their performing part and action category
  - A short description of the succession relationships (which actions must precede which) and at least one possible concurrent segment

Exercise 11.2: Decomposing an HVAC heating cycle into actions

Objective:

Define a decomposed action with nested sub-actions and explicit sequencing.

Estimated time: 25–30 minutes

Tasks:

- Design a **household HVAC heating system** consisting of:
  - A **Furnace** with a burner and heat exchanger
  - A **BlowerFan**
  - A **DuctNetwork** that distributes warm air
  - One **Thermostat** per floor (assume two floors)
- Define a single high-level behavior **HeatHouse,** that is performed by the **FurnaceController** and includes at least *five sub-actions*, such as:
  - Read thermostat setpoints.
  - Decide whether heat is required.
  - Ignite burner.
  - Start blower.
  - Monitor and shut down.
- *Decide which sub-actions must be strictly sequential, and which could be concurrent (for example, monitoring while heating)*.
- Identify where *values or items* (such as temperature readings or commands) must flow between sub-actions.
- Distinguish which sub-actions would naturally be *internal action usages* and which might be defined as *reusable action defs*.

Expected work products:

- A SysML package that includes:
  - At least *four part definitions* (Furnace, BlowerFan, DuctNetwork, Thermostat; *plus a* FurnaceController)
  - One *top-level action definition* HeatHouse with *named sub-actions*, each mapped to a performer
  - A clear *succession narrative* ("first read thermostats, then decide…") and at least one identified *concurrent behavior*
  - *An explicit list of data/items that flow between sub-actions (for example, temperature values, on/off commands)*

Exercise 11.3: Binding, defaults, and assignment in a power supply

Objective:

Differentiate use of bindings, default values, and assignment actions in a small cyber-physical component.

Estimated time: 25–35 minutes

Tasks:

- Model a **regulated DC power supply** with these parts:
  - **ACInputStage** (rectifier and filter)
  - **Regulator** (controls output voltage)
  - **MonitoringUnit** (measures output voltage and current)
- Define at least *three attributes* in the system:
  - A nominal reference voltage that should be an invariant binding (for example, 5 V)
  - A maximum current limit that has a default value but may be overridden by an assignment action
  - A current output voltage that changes over time and must be updated by assignment
- Describe one action that adjusts the regulator when the monitored output differs from the reference, using assignment semantics.
- Show when each of these values is:
  - Fixed by **binding**
  - Initialized by **default**
  - Modified by **assignment** during run-time behavior

Expected work products:

- A SysML package including:
  - Part definitions for ACInputStage, Regulator, MonitoringUnit, and a top-level PowerSupply
  - A list of attributes, each marked as *binding, defaulted, or assigned in an action*
  - A verbal description of an adjustOutput action that would:
    - Read monitored voltage.
    - Compare with the reference.
    - Use assignment to update the regulator control value.

Exercise 11.4: Send/accept actions in an elevator control protocol

Objective:

Design a message-based coordination protocol using send and accept actions between collaborating parts.

Estimated time: 35–45 minutes

Tasks:

- Consider a simple **elevator system** with these parts:
  - **CabController** (inside the elevator)
  - **FloorController** (for a single floor)
  - **DoorActuator**.
- Define at least three discrete signals/messages in the protocol, for example:
  - CallRequest from FloorController to CabController
  - DoorOpenCommand from CabController to DoorActuator
  - DoorClosedStatus from DoorActuator back to CabController
- For each message, decide:
  - Which part sends it (send action)
  - Which part accepts it (accept action), and
  - What payload attributes the message carries (for example, floor number, timestamp)
- Define the behavioral sequence for handling a call request and safely opening and closing the doors.
- Identify any continuous flows (for example, sensor values, position estimates) that may occur in parallel with the discrete messages.

Expected work products:

- A conceptual model of a package that would contain:
  - Part definitions for CabController, FloorController, DoorActuator
  - Named message usages with payload descriptions and sender/receiver mapping
  - A sequence narrative that could be turned into an action definition with send and accept actions and successions

Exercise 11.5: Conditional and looping behavior in an irrigation controller

Objective:

Apply if/else and loop actions to a realistic control problem.

Estimated time: 40–50 minutes

Tasks:

- Design a **garden irrigation controller** with parts:
  - ControllerUnit,
  - Three Valve parts (one for each zone: front yard, back yard, and garden)
  - MoistureSensorArray (one sensor per zone)

- Define a top-level behavior **RunIrrigationCycle** that:
  - Checks the time-of-day and only waters in an allowed window
  - Uses if / else logic to decide whether each zone needs watering based on moisture thresholds
  - Uses a loop (while or loop-until) to continue watering a zone until its moisture rises above a target
  - Optionally uses a for-each loop to iterate over all valves/sensors
- Decide which part(s) perform which actions, and where structured control actions live (for example, inside **ControllerUnit**).

Expected work products:

- A SysML package that contains:
  - Part definitions for **ControllerUnit**, **Valve**, and **MoistureSensorArray** (with multiplicities)
  - One **RunIrrigationCycle** action definition using:
    - At least one if/else decision
    - One loop over time
    - Optionally a for-each style iteration over zones
  - A prose account of how successions and conditions govern watering start/stop for each zone

Exercise 11.6: Concurrency with fork and join in a baggage-handling system

Objective:

Model concurrent behaviors and synchronization using fork and join control nodes.

Estimated time: 45–60 minutes

Tasks:

- Consider an **airport baggage-handling subsystem** with parts:
  - CheckInConveyor,
  - SecurityScanner,
  - Sorter (routes bags to gates)
  - JamDetector.
- Define a high-level behavior **ProcessBag** that:
  - Starts when a bag is placed on **CheckInConveyor**.
  - *Forks* into two concurrent actions:
    - Convey bag through **SecurityScanner.**
    - Monitor belt status via **JamDetector.**
  - Joins the flows before the Sorter routes the bag to its gate.
- Provide an alternative variant of the behavior *without explicit fork/join nodes*, using successions only, and analyze whether the intended concurrency is still clear.
- Identify which actions must complete before a bag can be declared "ready for loading".

Expected work products:

- A SysML system design including:
  - Part definitions for CheckInConveyor, SecurityScanner, Sorter, JamDetector
  - An action definition ProcessBag that includes:
    - A clearly described fork point
    - A concurrent segment
    - A join before sorting
  - A short comparison explaining how the model with fork/join versus without fork/join communicates concurrency and synchronization

Exercise 11.7: Swimlanes and perform actions in a drone delivery mission

Objective:

Map a multi-step behavior to performers using swimlanes and perform actions.

Estimated time: 60–75 minutes

Tasks:

- Model a **small drone delivery system** with parts:
  - **MissionController** (ground station)
  - **DroneAirframe** (includes navigation and propulsion)
  - **PayloadBay** (package handling)
  - **CommLink**.
- Define a mission-level behavior **ExecuteDeliveryMission,** that includes at least:
  - Receive mission plan.
  - Take-off and climb.
  - Navigate to destination.
  - Drop package.
  - Return and land.
  - Send mission status.
- For each sub-action, decide which part performs it and which are shared responsibilities (for example, mission planning versus onboard navigation).
- Express this as if you were going to draw a swimlane-style diagram:
  - One top-level action with nested sub-actions
  - A part hierarchy where **MissionController**, **DroneAirframe**, and **PayloadBay** perform different sub-actions via *perform actions*
- Identify at least one sub-action that is **reused** (performed in more than one context) and justify why an action def is appropriate there.

Expected work products:

- A conceptual SysML package containing:
  - Part definitions for MissionController, DroneAirframe, PayloadBay, CommLink, and a top-level DeliverySystem
  - An **ExecuteDeliveryMission** action with named sub-actions
  - An explicit mapping of **perform actions** from parts to these sub-actions (suitable for rendering as swimlanes)
  - A brief explanation of at least one reused action def and where it is performed

Exercise 11.8: Integrated interaction and action orchestration for a medical monitoring station

Objective:

Combine actions, flows, messages, structured control, concurrency, and perform actions into one cohesive scenario.

Estimated time: 75–90 minutes

Tasks:

- Design a **bedside medical monitoring station** with parts:
  - **PatientSensorSuite** (heart rate, SpO_2, blood pressure)
  - **MonitorController**,
  - **NurseDisplay**,
  - **AlarmUnit**,
  - **HospitalNetworkInterface**
- Define a top-level behavior **MonitorPatient** that includes at least:
  - Periodic acquisition of sensor data (looping behavior)
  - Conditional evaluation of alarm thresholds (if/else)
  - Concurrent logging of data to the HospitalNetworkInterface while updating the **NurseDisplay** (fork/join or equivalent)
  - Sending discrete alarm messages to **AlarmUnit** and network when thresholds are violated (send/accept + messages)
- Identify at least:
  - Five named actions
  - Two flows for continuous or structural data movement
  - Two message usages with explicit payloads (such as AlarmNotification, TrendSummary)
  - One explicit loop
  - One explicit decision
  - One concurrent segment
  - Two or more perform actions mapping behavior to structural parts
- Describe how the behavior starts (for example, patient connected) and ends (for example, monitoring session stopped or patient discharged).

Expected work products:

- A SysML package that includes:
  - Part definitions for all five system components
  - A fully described **MonitorPatient** action structure with:
    - Named sub-actions

    - Successions, decisions, and loops
    - At least one concurrency region with an implicit or explicit join
- Defined flows and message usages, including payload meanings and endpoints (structural for flows, event-based for messages)
- An explicit mapping of perform actions describing which part is responsible for which sub-actions

Chapter 12

Here is a progressive set of exercises to aid in your understanding and allow you to practice your coveted state machine skills.

Exercise 12.1: Automotive transmission state machine (introductory)

Estimated time: 10–20 minutes

Modern automatic transmissions operate through a small, clearly defined set of mutually exclusive modes: **Park**, **Reverse**, **Neutral**, **Drive**, and **Manual/Low Gear**. For this exercise, model the high-level state behavior of an automotive transmission control unit (**TCU**) that responds to driver input through a gear selector.

Problem description

The driver of a modern passenger vehicle can move the gear selector among several positions. Each position corresponds to a distinct operating mode of the transmission. The transmission control unit must reflect exactly one of these modes at any moment, and changing modes depends entirely on the physical selector position chosen by the driver.

You must model the behavior of the transmission control unit as a state machine. The transmission operates only in one mode at a time (no **concurrency**), and the transitions occur immediately when the driver moves the selector.

The modes you need to represent are:

- **Park**: Wheels are mechanically locked. The engine may be running or off.
- **Reverse**: The vehicle moves backward when the accelerator is applied.
- **Neutral**: No torque is applied to the wheels; the vehicle may coast.
- **Drive**: The vehicle moves forward with normal automatic gear selection.
- **LowGear**: A manually selected low gear for steep slopes or heavy loads.

When the selector is moved, the `Transmission` part receives an event indicating the newly selected position (such as `evSelectPark`, `evSelectReverse`, and so on). The TCU immediately switches to the corresponding mode.

Your task

Create a simple state machine for the `Transmission` with the following expectations:

- Create the structural elements that will exhibit the behavior. This includes the parts, items, ports, connections, attributes, and other structural aspects of the system necessary to support the desired behavior.
- Identify the states that correspond to the gear selector positions listed earlier.
- Identify the events the TCU receives when the driver changes the selector position.

- Define transitions between the states based solely on selector events.
- Include simple entry actions for each mode (such as enabling reverse lamps in Reverse, locking the drivetrain in Park, and so on).
- Ensure the state machine has a clear initial state. You may choose Park or another state, depending on your interpretation of the scenario.

No guards, nested states, or concurrent behavior are required for this exercise. Focus on a simple, direct mapping from driver commands to modal states.

Exercise 12.2: Medical thermometer

Estimated time: ~15 minutes

Problem description

A handheld digital medical thermometer has three basic behaviors:

- When powered on, it performs a quick self-check; if the self-check indicates no problems, then it becomes `Ready` for use; if the self-check fails, then it displays **"FAULT"** until the system is powered off or it reaches a low-power condition
- When ready, if the user presses the **Measure** button, the thermometer enters measuring mode, where it samples temperature repeatedly until it detects a stable reading (five measurements in a row result in the same value within 0.5 °C)
- Once a stable reading is obtained, it enters a mode where it displays the measured temperature for 20 seconds and then automatically returns to the `Ready` condition
- The user can press the **Power Off** button regardless of the mode, and the device will turn off
- If the battery level drops below a threshold during operation, the thermometer immediately enters a low-battery condition, where it flashes a warning for 60 seconds and then powers itself off

Tasks

- Create the structural elements that will exhibit the behavior
- Identify a set of states (expect approximately 4–8)
- Identify events (button presses, significant value changes, timeouts, and so on)
- Identify which states might use entry/do/exit actions
- Create the state machine, showing initial behavior after `PowerOn`

Exercise 12.3: Home alarm system (basic reactive, guards)

Estimated time: approximately 20–25 minutes

Problem description

A home alarm system has these behaviors:

- When first powered, it goes to a `disarmed` condition where it does no monitoring or alarming.

- When the user enters a correct **PIN**, the system allows the user to select between operational modes: `ArmedHome` (only certain areas are monitored) or `ArmedAway` (all areas are monitored).
- If a door or window sensor trips while in `ArmedAway`, the alarm immediately enters *a preparing-to-alarm* state.
- If a motion sensor trips while in `ArmedHome`, the alarm enters `Triggered` *only if* the tripped zone is not marked as an *interior safe zone*.
- After entering the `preparing-to-alarm` state, a countdown begins: if the user enters a valid PIN within 120 seconds, the system returns to the `disarmed` condition and monitoring is stopped; otherwise, the system transitions to an `alarming` state in which the siren sounds.
- The `alarming` state continues for 10 minutes or until you disable it by entering a valid PIN. If you do not enter a valid PIN, then the system turns off the siren and returns to the previous monitoring condition. If you enter a valid PIN, the system goes to the `disarmed` condition.

Tasks

- Create the structural context for the model, including all necessary features (such as attributes and items)
- Identify states, triggers, and transitions, including any appropriate guards
- Create the state machine(s)
- Add the actions to support the specified behaviors
- Sequence the actions within the states as entry, do, exit, and transition actions

Exercise 12.4: Medical ventilator (inspiratory/expiratory cycling)

Estimated time: ~25–35 minutes

Problem description

A simplified ventilator cycles between phases:

- After initialization, it enters a standby condition in which it moves no gas
- When the clinician presses the **StartVentilation** button, the ventilator enters the `Ventilating` state, which is a composite state with two nested phases:
  - `Inspiratory` (delivers air for a configurable inspiratory time)
  - `Expiratory` (allows passive exhalation for a configurable expiratory time)
- Transitions between `Inspiratory` and `Expiratory` occur purely based on relative-time triggers
- If airway pressure exceeds a safety threshold at any point, the ventilator leaves Ventilating and enters `PressureAlarm`
- When the alarm clears, and the clinician presses **Reset**, the ventilator returns to `Standby`
- At any time during ventilation, the clinician may press the **Pause** button, which suspends the breathing cycle in a `Paused` state until the clinician presses the Pause button again
- At any time during ventilation, the clinician may press the **StopVentilation** button, and the system returns to the standby condition

Tasks

- Create the structural context, including the attributes for the inspiratory and expiratory times and the commanded gas flow rate during the inspiratory phase
- Identify the states, including the composite state and its nested states
- Identify all event triggers (**StartVentilation**, time triggers, and so on) and add them as triggers to the transitions
- Add entry, exit, do, and transition actions as appropriate to control the gas flow and valves

Exercise 12.5: Traffic light controller (two-road intersection)

Estimated time: 60–90 minutes

Problem description

This exercise asks you to build a realistic state-machine-based controller for a four-way intersection. The intersection contains a primary road (higher-volume) and a secondary road (lower-volume). Each road has two types of lanes:

- Through lanes (one in each direction)
- Left-turn lanes (one in each direction)

Both roads are instrumented with *buried passive loop inductors* installed in each left-turn lane. These sensors detect the presence of vehicles waiting to turn left and send this information to the controller. The through lanes do not use sensors; they operate solely on a timed cycle.

A central `TrafficController` part coordinates four sets of traffic lights, one for each of:

- `Primary-Road-Through`
- `Primary-Road-Left-Turn`
- `Secondary-Road-Through`
- `Secondary-Road-Left-Turn`

Each set holds two traffic lights, one for each direction (for example, one for traffic going north and one for traffic going south on the same road). The lights in a set must always be controlled simultaneously (for example, north-south traffic flows at the same time while east-west traffic waits).

Behavioral requirements (fixed-cycle mode)

Your state-machine model must implement the following rules:

Through-lane rules (time-driven)

1. Each road has a 15-second `GREEN` phase and a 5-second `YELLOW` phase. After that road turns `RED`, the intersection remains all-red for 2 seconds before a conflicting road's permitted phase begins. A road remains `RED` throughout the opposing road's permitted phases.
2. When a road's through lanes are `GREEN` or `YELLOW`, the other road's through lanes must remain `RED`
3. After a road's lights turn `red`, all lights are `Red` for 2 s
4. Through lanes for both directions on the same road are controlled together; they always show the `same aspect` at the same time (for example, east-going and west-going traffic on the same road travel at the same time)

Left-turn lane rules (sensor-driven)

1. Left-turn lanes remain `RED` unless a car arrives at the loop inductor in the turn lane. Cars may arrive at any time in the cycle but will only get a `green` light at appropriate times.
2. Detecting a vehicle in either left-turn lane on a road *requests* a turn phase for that road.
3. A requested turn phase *does not start immediately*. Instead, it waits until the *next* moment when that road's through lanes would normally turn GREEN.
4. When the activation time arrives and there is at least one car waiting to turn left, both left-turn lanes on that road run a full `RED`–`GREEN`–`YELLOW`–`RED` cycle.
5. During this cycle, the corresponding through lanes must remain `RED`.
6. Once the turn phase completes, the controller transitions to the road's normal through-lane `GREEN` cycle.
7. After the left-turn cycle has run once, the turn request is considered *handled*, and another vehicle arrival is required to activate a future turn phase.

These rules require the integration of time-based triggers, event-based triggers, guards, and nested sub-state behavior. They also require cooperation between multiple parts representing the sensors and signal heads.

Your modeling tasks

Model the structural and behavioral aspects of the intersection using SysML v2:

Structural model (parts)

Define at least the following elements:

- A `TrafficController` part responsible for state behavior
- `TrafficLight` parts (or equivalent) for each of the four groups:
  - `PrimaryThrough`, `PrimaryLeft`, `SecondaryThrough`, and `SecondaryLeft`
- `LoopSensor` : One part for each road, representing the two physical left-turn sensors on that road as a single logical element for modeling purposes (two `LoopSensor` parts total):
  - `PrimaryLeftSensor` and `SecondaryLeftSensor`
- Connections or bindings between sensors and the controller so that the controller can accept events indicating vehicle arrival

You may treat each signal group as a part with actions to set `RED`, `GREEN`, and `YELLOW`, or treat them as simple data attributes updated by controller actions.

Behavioral model (state machines)

Within the `TrafficController`'s state machine:

- Define a composite OR-state per road (`PrimaryControl` and `SecondaryControl`), each containing:
  - A fixed-cycle `ThroughPhase` (with timed `GREEN`, `YELLOW`, and `RED` substates)
  - A `TurnPhase` with its own `RED`–`GREEN`–`YELLOW`–`RED` sequence
  - A mechanism for prioritizing `TurnPhase` when activated by a sensor
- Use *value-based guards* or explicit events to ensure:
  - A left-turn phase begins *only* when:
    - A turn request is pending
    - The through-phase for that road would normally begin
  - After completing a turn phase, the request flag is cleared
- Use relative-time triggers for `GREEN`, `YELLOW`, and `RED` timing intervals
- Use message-based triggers from sensors to capture turn-lane arrivals
- Use null-triggered transitions if appropriate for sequencing within a fixed timed cycle
- Ensure the two roads never display `GREEN` (or `YELLOW`) simultaneously

You may structure this either as a single top-level state machine coordinating both roads or as two road-level machines operating under a coordinating master state.

Exercise 12.6: Hospital infusion pump (guards and safety conditions)

Estimated time: ~40 minutes

Problem description

An infusion pump administers **IV** fluids at a prescribed rate:

- After powering on and self-test, it enters a `Ready` condition
- From `Ready`, if the nurse presses the **SetUp button**, the system enters `ProgramMode`, where a nurse enters a dosage rate and volume
- When the nurse presses **Start**, the pump enters `Infusing` (active state)
- If the line occludes (pressure rises sharply), it enters `OcclusionAlarm`
- If the bag becomes empty, it enters `EmptyBagAlarm`
- If the clinician presses **Pause**, infusion stops and the pump enters `Paused`
- From `Paused`, pressing **Start** resumes infusion *only if*:
  - A valid prescription remains
  - The bag is not empty
- If the volume infused reaches the programmed amount, the pump enters Complete, in which it displays the amount of drug injected
- From `Complete`, the nurse can press the **SetUp button** to reenter `ProgramMode`
- At any time, the **Off** button turns the system off

Tasks

- Create the structural elements and their structural properties
- Identify states, events, and transitions
- Identify guards where appropriate
- Identify actions and add them as entry, do, exit, or transition actions

Exercise 12.7: High-energy laser engagement controller

Concepts: Multi-part coordination, sensor-triggered transitions, warm-up and cooldown timing, composite states, guarded transitions, inter-part messaging, and safety interlocks.

Estimated time: 60–90 minutes

Problem description

High-energy laser (**HEL**) systems are increasingly used for threat suppression and precision engagement. A laser weapon differs from conventional kinetic systems in that the beam can be steered, fired, or aborted extremely quickly, but only if the laser subsystem is in the correct thermal and electrical state. This exercise focuses on modeling the state behavior of a simplified HEL engagement controller and its collaboration with a separate `TargetTracker` part.

A ground-based vehicle mounts a high-energy laser turret. The system has two major behavioral components:

- `TargetTracker`
  - Continuously scans for objects
  - When an object of interest is detected, it emits a `TargetAcquired` event
  - If the target moves out of track lock, it emits a `TrackLost` event
- `LaserWeaponController`
  - Manages the internal state of the HEL weapon
  - Must perform a warmup process before firing
  - Continuously monitors system temperature; overheating forces an automatic `CoolDown` state
  - Fires only when:
    - A target is tracked
    - The laser is warmed up
    - No safety interlocks are active
    - A `StartFiring` command is received

The weapon has several relevant timing constraints:

- `WarmUpTime` (for example, `4 s`): Time required to charge, stabilize optics, and spin up thermal pumps
- `FiringBurstDuration` (for example, `1.5 s`): Maximum allowable continuous fire before mandatory cooldown
- `CooldownPeriod` (for example, `6 s`): Required cooling cycle before another warmup may begin

A `SafetyInterlock` signal may be raised at any time (such as an open hatch, detected personnel, turret misalignment, or when there is a satellite in orbit behind the target). When active, the laser must *immediately* cease firing and transition to a safe state.

Structural elements to include

Define at least these parts:

- `LaserWeaponController` (main state machine)
- `TargetTracker` (produces `TargetAcquired` and `TrackLost` events)
- `ThermalMonitor`
  - Reports `Overheat` when the internal temperature exceeds the threshold
  - Reports `TemperatureNormal` when safe
- `SafetySystem`
  - Emits `SafetyInterlockOn` and `SafetyInterlockOff` events

The controller must accept events from the other three collaborators.

Behavioral requirements

Your state-machine model for `LaserWeaponController` must satisfy the following:

1. `Idle` and `warmup`
   - Initial state is `Idle`
   - When a `TargetAcquired` event arrives, transition to `WarmUp`
   - `WarmUp` completes after a fixed duration:
   - `accept after WarmUpTime` ⟶ `Armed`

2. `Armed`/ready-to-fire
   - In `Armed`, the weapon waits for the tracker to indicate continuous lock
   - If `TrackLost` occurs, return to `Idle`
   - If `SafetyInterlockOn` occurs, transition immediately to `SafeHold`

3. `Firing`
   - When `Armed` and receiving a `StartFiring` event, transition to `FiringBurst`
   - In `FiringBurst`:
     - A do-action performs beam output
     - After `FiringBurstDuration`, transition to `CoolDown`
     - If `TrackLost` or `SafetyInterlockOn` occurs at *any* time, cease fire immediately and go to `SafeHold`

4. `Cooldown`
   - In `CoolDown`, no firing is allowed
   - After `CooldownPeriod`, return to `Idle` unless a target is still tracked, in which case return to `WarmUp` automatically

5. `SafeHold`
   - Activated by any safety violation
   - Remains in this state until `SafetyInterlockOff` occurs
   - Then returns to `Idle`

Your modeling tasks

- Create the structural elements and their features, including attributes, parts, and items
- Identify all states needed for the `LaserWeaponController`
- Define transitions driven by message events, timeouts, or change events
- Add guards where appropriate (for example, preventing firing if thermal conditions are unsafe)

- Define the collaboration pattern: how the controller accepts events from each collaborating part using appropriate `send` and `accept` actions
- Consider whether any states should contain internal do-actions (such as continuous firing or thermal pump operation)
- Identify and add necessary actions to the states with appropriate sequencing

Exercise 12.8: Multi-room smart lighting (concurrent/AND states)

Estimated time: ~60 minutes

Problem description

A smart home lighting controller manages lighting in two independent rooms: `LivingRoom` and `Kitchen`. Each room has identical modes:

- `Off`
- `On`
- `Dimmed`

Users can issue separate commands to each room. Additionally:

- A global `AwayMode` turns *both* rooms Off regardless of their previous state
- When returning from `AwayMode`, each room resumes its prior state
- Motion detection in either room temporarily turns that room `On` (overriding `Dimmed`) for 2 minutes, after which it returns to its pre-motion state
- The controller must be modeled using an AND-state: one region per room
- `AwayMode` is a top-level state that overrides both regions

Tasks

- Create the structural context, including all relevant properties
- Identify the top-level states and the two concurrent regions
- Identify events that cause state changes
- Define how each region restores its previous state when exiting `AwayMode`
- Add and sequence actions as appropriate
- Describe in prose how simultaneous motion in both rooms is handled under RTC semantics

Exercise 12.9: Train door controller (safety guards and concurrency)

Estimated time: ~75–90 minutes

Problem description

A passenger train has doors controlled by a central system:

Top-level behavior:

- `Locked` (train moving or not at station)
- `Unlocked` (train at station and ready for boarding)
- `Emergency` (manual emergency override or critical system-wide safety fault)

Inside `Unlocked`, each door (`DoorA` and `DoorB`) is modeled as its own region in an AND-state.

Each door has states:

- `Closed`
- `Opening`
- `Open`
- `Closing`

Behavior rules:

- Doors may begin `Opening` only when the top-level state is `Unlocked`
- An obstruction sensor can fire while `Closing`, forcing that door to reopen
- If a manual override handle is pulled on either door, the top-level state immediately enters `Emergency`, and both doors must open and remain open until the `Emergency` condition is removed (manual override handle is restored to its default position)
- When the conductor commands `CloseAll`, each door begins `Closing`, but a door may not close if a `PassengerDetected` sensor detects an obstruction
- When the train leaves the station, the system must transition to `Locked` only when both doors are closed

Tasks

- Create the structural context and all relevant properties
- Identify top-level states and nested concurrent regions
- Identify all events, transitions, and guards
- Elaborate the entire state machine (top-level plus two concurrent regions)
- Add and sequence actions with entry, do, exit, and transition actions
- Describe where race conditions *might* appear if not carefully structured

Exercise 12.10: Autonomous warehouse robot (complex nesting, concurrency, and guards)

Estimated time: 90 minutes

Problem description

A warehouse robot moves material within a warehouse. It performs cycles of navigation and payload handling.

Top-level states:

- `Idle`
- `Navigating`
- `HandlingPayload`
- `Charging`
- `Fault`

Inside `Navigating`, the robot has a composite state with nested states:

- `GoingToPickup`
- `GoingToDropoff`
- `AvoidingObstacle` (entered only when an obstacle is detected; returns to previous nested state when clear)

Inside `HandlingPayload`, the robot has an AND-state with two concurrent regions:

- Fork state machine (`Raise`/`Lower`/`Locked`)
- Safety interlock machine (`Engaged`/`Disengaged`)

Additional behavior requirements:

- Transition from `Navigating` to `HandlingPayload` only when the robot has reached the pickup or drop-off location and the safety interlock is `Engaged`
- If the battery level falls below a threshold during any operation, the robot must transition to `Charging`, but only after completing its current RTC step
- From `Charging`, the robot returns to `Idle` when the battery reaches a sufficient level
- Any critical sensor fault causes an immediate transition to `Fault` regardless of nested or concurrent states
- In `AvoidingObstacle`, upon obstacle clearance, the robot returns to whichever nested state it previously occupied (you must track this)

Tasks

- Create the structural context with all parts and relevant properties
- Identify all top-level states, nested states, and parallel states
- Identify events and guards (obstacle detected, obstacle cleared, battery low, location reached, interlock engaged/disengaged, etc.)
- Add and sequence actions within the states and transitions
- Describe how the return-to-previous-state behavior works in `AvoidingObstacle`
- Create the entire multilevel state machine structure
- Describe potential race conditions and how modeling patterns could avoid them

Chapter 13

These exercises are here for you to hone your skills with the concepts discussed in this chapter.

Exercise 13.1 — solar panel area and power estimate

Estimated time: 15–20 minutes

Problem description

A small off-grid cabin is powered by a single rectangular solar panel mounted on the roof. The installer wants to model the panel's basic geometric and power properties in SysML v2 so that the design can be reused for different panel sizes.

Tasks

Each panel is characterized by:

- **width** and **height** (panel dimensions)
- **irradiance** (solar power per unit area)
- **efficiency** (fraction of incoming power converted to electricity)
- Define a `calc definition` that computes the *area* and *nominal power* of a panel from width, height, irradiance, and efficiency. Use:
  - At least one local attribute for an intermediate expression, such as area
  - A single return value (such as `nominalPower`)
- Create a part definition for `SolarPanel` with attributes width, height, irradiance, and efficiency.
  - Use binding (`=`) for one attribute that is always equal to a mathematical expression of the others (such as area)
  - Inside an action, use assignment (`:=`) to update an attribute (such as `currentPower`) from the calculation result
- Write one or two relational constraints ensuring that:
  - Width and height are strictly positive
  - efficiency is between 0 and 1 (inclusive), using logical operators (`and`)

Focus: basic calc definition and usage, binding versus assignment, arithmetic, relational, and logical operators.

Exercise 13.2 — elevator load and safety logic

Estimated time: 20–30 minutes

Problem description

A building elevator has a maximum allowable load. The elevator controller monitors the *number of passengers* and the *approximate average mass per passenger* to estimate total mass. It must prevent the doors from closing if the estimated mass exceeds the rated capacity.

Tasks

- Define a `calc definition` that:
  - Takes parameters: `numPassengers : Integer`, `avgMass : Real`, and `maxCapacity : Real`
  - Returns a `Boolean` indicating whether the load is within safe limits
  - Uses arithmetic and relational operators (`*`, `<=`) and at least one logical operator (`and or or`) in the final expression
- Create a part definition `ElevatorCab` with attributes:
  - `numPassengers`, `avgMass`, `maxCapacity`, and a derived `estimatedMass` using binding
  - A `generic constraint` indicating that `numPassengers` must be non-negative
- Add an `assert constraint` on the `ElevatorCab` part that must always be true during operation, such as:
  - The elevator must never operate with a negative mass or more than some hard upper bound
- In an action, show how the controller:
  - Uses an implicit calculation usage to determine whether `doorsMayClose` should be set true or false via assignment (`:=`)

Focus: Boolean calculations, relational and logical operators, generic versus assert constraints, implicit calc usage.

Exercise 13.3 — battery pack health index (RealFunctions)

Estimated time: 25–35 minutes

Problem description

An electric scooter has a small battery pack with several monitored parameters: *state of charge*, *number of cycles*, and *internal resistance*. A health index is calculated to estimate the remaining usable life, normalized to the range [0, 1].

Tasks

- Define a `calc` definition `BatteryHealthIndex` that:
  - Uses in parameters such as `stateOfCharge`, `cycleCount`, and `internalResistance`
  - Uses at least two functions from `RealFunctions`, such as `abs`, `max`, `min`, `sqrt`, or `sum`
  - Returns a normalized `Real` between 0 and 1 (use relational and logical operators to enforce or clamp)
- In a `BatteryPack` part:
  - Declare attributes for the monitored quantities
  - Add an attribute `healthIndex` that is bound using = to the result of `BatteryHealthIndex(...)`

- Introduce an `assert constraint` on the `BatteryPack` part to ensure:
  - `healthIndex` is always between 0 and 1
- Create a small action that:
  - Uses assignment to copy `healthIndex` into another attribute `displayedHealth` only if some condition holds (for example, when the system is powered)
  - Uses a logical operator (`and`, `or`, `not`) in this condition

Focus: `RealFunctions` usage, binding to calcs, assert constraints, combined arithmetic and logical expressions.

Exercise 13.4 — drone turning geometry (TrigFunctions)

Estimated time: 30–45 minutes

Problem description

A quadcopter drone flies at constant speed and turns by banking with a roll angle. A simplified flight model relates speed, turn radius, gravitational acceleration, and bank angle. The drone's controller must compute the required bank angle to achieve a desired turn radius and ensure it remains within safe limits.

Tasks

- Import `TrigFunctions` and define a calculation that:
  - Accepts parameters such as `speed : Real`, `turnRadius : Real`, and `g : AccelerationValue` or `Real`
  - Uses at least `tan`, `atan`, `rad`, or `deg` to compute a *bank angle* (in radians or degrees)
  - Uses a local attribute for an intermediate quantity (e.g., **load factor**)
- Model a `DroneFlightController` part with:
  - Attributes for `speed`, `desiredTurnRadius`, `maxBankAngle`, and a bound attribute `commandedBankAngle`
  - A generic constraint to indicate that `turnRadius` and `speed` must be positive
- Add an assert constraint that:
  - Ensures `commandedBankAngle` never exceeds `maxBankAngle`
- Create a requirement (separate element) that:
  - States a performance limit on bank angle or turning radius
  - Includes a require constraint capturing that limit formally

Focus: TrigFunctions, unit/angle conversions, calc usage in a controller, assert and require constraints.

Exercise 13.5 — fast charging requirement with assume/require

Estimated time: 25–35 minutes

Problem description

An EV manufacturer specifies a requirement for *fast charging* a vehicle battery. The charging station must respect both a *maximum charge current* and a *temperature envelope* for safe fast charging.

Tasks

- Create a requirement `FastChargeSafety` with in parameters such as:
  - `cellTemp : TemperatureValue`
  - `chargeCurrent : ElectricCurrentValue`
  - `ratedCapacity : ElectricChargeValue`
- Inside the requirement, define:
  - An assume constraint capturing the *ambient or cell temperature range* in which the requirement applies
  - A require constraint that limits the allowable charge current (for example, `chargeCurrent` ≤ some fraction of `ratedCapacity`)
- Add a short natural-language description (in the requirements doc property) that explains the logical meaning:
  - If the assumed temperature range is satisfied, then the required current limit must hold
- Optionally, define a small calc definition that computes a recommended maximum charge current from temperature and rated capacity, and use it inside the require constraint

Focus: assume versus require constraints, requirements as formal Boolean conditions, optional calc inside a requirement.

Exercise 13.6 — water tank level and safety invariants (assert and generic constraints)

Estimated time: 20–30 minutes

Problem description

An industrial process uses a vertical water tank with a **level sensor**, **inlet valve**, and **overflow sensor**. The controller must compute a safe operating range and prevent overflow under normal conditions.

Tasks

- Define a `Tank` part with attributes:
  - `level : LengthValue`
  - `maxLevel : LengthValue`
  - `minLevel : LengthValue`
  - `overflowDetected : Boolean`
- Add a generic constraint (non-asserted) expressing something like:
  - `minLevel < maxLevel and minLevel >= 0`

- Add an assert constraint that:
  - States that under normal operation (for example, `overflowDetected == false`), the tank level must always be ≤ `maxLevel`
- Define a small calculation that computes a normalized fill fraction (0 to 1) from `level`, `minLevel`, and `maxLevel`
  - Use this calculation in a bound attribute `fillFraction` using =
  - Add a constraint ensuring `0 <= fillFraction <= 1` (use relational and logical operators)

Focus: generic vs assert constraints on parts, simple calc and binding, expression language with relational and logical operators.

Exercise 13.7 — EV range estimator and unit consistency

Estimated time: 40–50 minutes

Problem description

An EV model includes a **range estimator** that predicts how far the vehicle can travel given the current battery state of charge, average energy consumption, and driving speed. Unfortunately, different engineering teams have used different units and quantity types in their models.

Tasks

- Create a part definition `EVRangeEstimator` that initially uses plain `Real` attributes for:
  - `batteryEnergy` (intended kWh)
  - `energyPerDistance` (intended kWh per 100 km)
  - `estimatedRange` (intended km)
- Write a calc definition that computes `estimatedRange` from these values but *intentionally mixes units* (for example, treating per 100 km as per km) so that the result is numerically wrong but dimensionally invisible when using `Real`
- Now refactor the model to use explicit quantity kinds and units:
  - Use appropriate quantity types from SI/ISQ (e.g., energy and distance)
  - Update the calculation so that the dimensional inconsistency becomes obvious
- Introduce a second calc using `QuantityCalculations::ConvertQuantity` to convert all values to a consistent set of units (for example, kilometers and kilowatt-hours) before computing range
- Add a short generic constraint or assert constraint expressing that:
  - The estimated range must always be ≥ 0 and less than some plausible upper bound (for example, 2,000 km)

Focus: implicit dimensionality with plain `Real` (hidden errors), explicit quantity kinds, unit conversion with `ConvertQuantity`, constraints on results.

Exercise 13.8 — power bus computation with explicit calc usages

Estimated time: 45–60 minutes

Problem description

An avionics system has several electronic loads connected to a shared 28 V DC power bus: radios, displays, and computers. The system architect wants to compute total bus power, per-device power, and safety margins using reusable calculations.

Tasks

- Define a **calc** definition **ComputePower** that:
  - Takes in parameters **voltage** : `VoltageValue` and **current** : `ElectricCurrentValue`
  - Returns **voltage * current**
- Create a **part** definition **AvionicsDevice** with:
  - **supplyVoltage** : `VoltageValue`
  - **supplyCurrent** : `ElectricCurrentValue`
  - An attribute **devicePower : PowerValue** bound via an explicit calculation usage that returns the product of the device's voltage and current (typed as `VoltageValue` and `ElectricCurrentValue`)
- Define a **PowerBus** part that aggregates multiple **AvionicsDevice** parts
  - Use either a `sum` function (from **RealFunctions**) or a custom calc to compute total bus power from the devices' **devicePower**
  - Create a generic constraint or assert constraint that **totalBusPower** must not exceed a given **busRating**
- Demonstrate:
  - One implicit usage of **ComputePower** (direct call)
  - One explicit usage via a named calculation usage that specializes or redefines a parameter (for example, defaulting the bus voltage to 28 V)

Focus: implicit versus explicit calc usage, specialization/redefinition of parameters, aggregation and summation, assert/generic constraints.

Exercise 13.9 — climate chamber with defaults, constraints, and a simple textual body

Estimated time: 45–60 minutes

Problem description

A test **climate chamber** can be configured for different temperature/humidity profiles. The model should compute a comfort index, enforce limits, and (optionally) use an external snippet to compute a more complex function not easily expressed in pure SysML.

Tasks

- Model a **ClimateChamber** part with attributes:
  - **targetTemp** : `TemperatureValue` with a default bound value (for example, 22 °C)
  - **targetHumidity** : `Real` with a default initial value (default :=)
  - **comfortIndex** : `Real` bound to a calculation using arithmetic, relational, and logical operators (for example, penalizing too hot/too cold and too dry/too humid)
- Add:
  - A generic constraint that states the allowed ranges for **targetTemp** and **targetHumidity**
  - An assert constraint that the computed **comfortIndex** must remain within a specified [min, max] range during operation
- Define a requirement **ComfortProfileReq** that:
  - Has an assume constraint describing the applicability conditions (for example, when a certain mode is active or when external conditions fall in a range)
  - A require constraint formalizing a minimum acceptable comfortIndex
- (Optional) Add a second calc for **AdvancedComfortMetric** whose body is provided via a textual representation (for example, rep language "Groovy"), calling a more complex external function. Use it in parallel with the pure SysML calculation and add a constraint relating the two (for example, they must not differ by more than some tolerance).

Focus: default binding versus default initial value, calc definitions with mixed operators, all four constraint kinds, and (optionally) a simple textual representation body tied back to a SysML-declared signature.

Chapter 14

The following exercises illustrate how SysML v2 *cases*—including generic cases, analysis cases, trade studies, and verification cases—are applied in practice. Each example is deliberately small in scope but representative of common engineering situations. The intent is not only to show correct syntax, but to demonstrate the reasoning patterns, structural organization, and modeling conventions that allow cases to serve as first-class analytical and verification elements within a system model.

Exercise 14.1 — Generic case for a water tank scenario

Estimated time: 15–20 minutes

Goal

Create a generic case that evaluates an operational interaction and returns a meaningful performance metric.

Tasks

- Consider a water-storage system with a tank, a fill valve, and an external water supply
- Create a *generic case that computes the predicted fill time* for the tank to reach its target level
  - Math/processing: Fill time is computed as: `fill time = required volume/inflow rate`, where inflow rate is constrained by the actor (external supply)
- Identify the subject and actor, and create an objective constraining the actor's flow rate
- Explain why this is a generic case (the objective constrains the actor rather than the returned fill time)

Exercise 14.2 — Generic case for robotic arm load handling

Estimated time: 15–20 minutes

Goal

Model a generic case that evaluates external constraints while returning a system-level operational outcome.

Tasks

- Consider a robotic arm that receives a load from an operator and places it on a target platform
- Create a generic case that computes the **placement accuracy**, that is, the deviation between the commanded target pose and the measured final pose
  - Math/processing: Placement accuracy may be computed using a Euclidean distance or angular deviation metric, such as:
  - $\texttt{pos error = sqrt\{(x}_c - x_a)^2 + (y_c - y_a)^2 + (z_c - z_a)^2\}$, where a refers to the actual position and c refers to the commanded position.
- Identify the subject and actor, and give an objective constraining the actor (for example, load mass)

Exercise 14.3 — Analysis case for electronics thermal dissipation

Estimated time: 20–25 minutes

Goal

Structure an analysis case where the computed result is evaluated against a limit.

Tasks

- Consider an electronics enclosure cooled by a heat sink
- Create an analysis case that computes the predicted steady-state internal temperature
  - Math/processing: A simple thermal-resistance model computes temperature as:
  - $T_{steady} = T_{ambient} + (P * theta)$, where P is power dissipation and theta is thermal resistance.
- Define the inputs and create an objective that evaluates the result against a temperature limit

Exercise 14.4 — Trade study analysis case for drone battery selection

Estimated time: 30–40 minutes

Goal

Apply the trade-study workflow to evaluate alternatives and select the best one.

Tasks

- Consider three drone batteries with different capacities, masses, and costs
- Create a *trade study that selects the battery alternative with the highest weighted utility score*
  - Math/processing:
    - Use a linear utility curve for each criterion:
    - $U = 10 \times (value - worst) / (best - worst)$.
    - Compute a weighted sum:
    - $S = w_c U_c + w_m U_m + w_{cost} U_{cost}$
    - where all weights sum to 1.0.
    - The selected alternative is the one with the maximum score.
- Define best/worst values and weighting factors

Exercise 14.5 — Verification case for pressure sensor accuracy

Estimated time: 25–30 minutes

Goal

Develop a verification case that computes evidence and returns a verdict.

Tasks

- Create a requirement that a pressure sensor must report pressure within ±2% of true pressure
- Create a system that satisfies that requirement
- Create a verification case that computes the measured pressure error and returns a verdict
  - Math/processing:
  - Compute the measurement error:
  - $error = |P_{reported} - P_{true}|$.
  - Compute the allowed tolerance:
  - $max\ error = 0.02 * P_{true}$.
  - Verdict is pass if $error \leq max\ error$.
- Identify the subject, test input, and verification action that performs the measurement
- Include an objective linking the case to the requirement

Exercise 14.6 — Hierarchical verification case for a smart light controller

Estimated time: 30–40 minutes

Goal

Construct a top-level verification case that aggregates subordinate cases.

Tasks

- Create a smart light controller that satisfies two requirements:
  - On-command response time ≤ 200 ms
  - The smart light controller shall maintain the actual brightness within ±5 percentage points of the commanded brightness, where both are expressed as percentages of maximum brightness (0–100%)
- Create two subordinate verification cases:
  - One *computes the measured turn-on time*.
  - Math:
  - $t_{response} = t_{reported\ on} - t_{command}$.
  - The other *computes dimming error*.
  - Math:
  - $error = |B_{actual} - B_{command}|$.

    - Each subordinate case returns a `pass/fail` verdict based on whether its computed value satisfies its requirement's constraint
    - Create a top-level case that returns `pass` only if both subordinate verdicts are `pass`